

Sistema de Controlador RC por Bluetooth

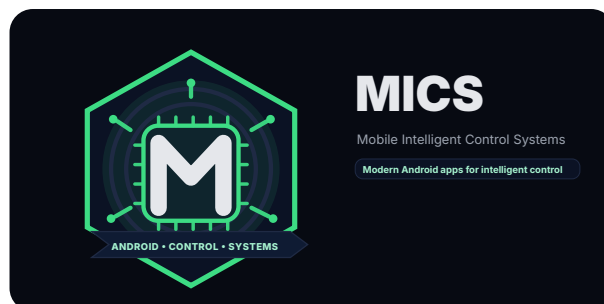
Documentación de la App Android + Firmware ESP32

(EmitterApp + ESP32_BT_Controller)

Este documento explica cómo construir, cablear, configurar y depurar un sistema de control remoto basado en Bluetooth utilizando una aplicación Android y un firmware controlador en ESP32.

Temas incluidos

- Carga del firmware y conexión Bluetooth
- Controles básicos (joysticks, perillas, interruptores, botones)
- Diagramas de cableado y asignación de pines (`pins.h`)
- Paneles, indicadores y gráficas de telemetría
- Capa de control vs abstracción de hardware (`control.cpp`, `hal_output.cpp`)
- Solución de problemas (Android y ESP32)



Fecha: 15 de mayo de 2026

Índice general

1. Introduction	6
1.1. What is the ESP32 RC Controller System	6
1.2. Who This Guide is For	6
1.3. What You Can Build	7
2. ESP32 Code Architecture Overview	8
2.1. High-Level Architecture (Android App ↔ ESP32)	8
2.2. Initialization Layer (Diagram: <i>Initialization</i>)	8
2.2.1. Serial Initialization (<code>serialInit()</code>)	8
2.2.2. Bluetooth Initialization (<code>bluetoothInit()</code>)	8
2.2.3. Hardware Initialization (<code>hardwareInit()</code>)	9
2.3. Runtime Layer (Diagram: <i>Runtime / main loop</i>)	9
2.4. Bluetooth Receive Subsystem (Diagram: <i>Bluetooth RX (receiver)</i>)	9
2.4.1. Input Packets	9
2.5. Shared Data Model (Diagram: <i>packets.h</i>)	10
2.6. Control Layer (Diagram: <i>Control layer</i>)	10
2.6.1. Event Handling	10
2.7. Hardware Abstraction Layer (Diagram: <i>HAL outputs</i>)	10
2.8. Optional GPIO Expansion (Diagram: <i>MCP23017 (optional)</i>)	11
2.9. Telemetry Subsystem (Diagram: <i>Telemetry TX and Telemetry sources</i>)	11
2.9.1. Connection-Aware Behavior	11
2.9.2. Telemetry Sources	11
2.10. Debug Subsystem (Diagram: <i>Debug</i>)	11
2.11. Pulse Subsystem (Diagram: <i>Pulse outputs</i>)	12
2.12. Summary: Why the Architecture is Structured This Way	12
3. Hardware Setup	14
3.1. ESP32 Development Board	14
3.2. Control Outputs (What the App Controls)	14
3.3. Motor Driver Connection	15
3.4. Servo Connections	15
3.5. Optional Inputs and Telemetry Sensors	15
3.6. Power Supply and Safety	16
3.6.1. Recommended power arrangement	16
3.6.2. Grounding	16
3.6.3. Practical notes	16
4. Uploading the Firmware	17

4.1.	Install the Arduino IDE	17
4.2.	Install ESP32 Board Support	17
4.3.	Download the Firmware	18
4.4.	Open the Firmware Project	18
4.5.	Select the ESP32 Board	18
4.6.	Configure Board Parameters	18
4.7.	Connect the ESP32 Board	19
4.8.	Compile the Firmware	19
4.9.	Upload the Firmware	19
4.10.	Verify Firmware Operation	20
4.11.	Next Step	20
5.	Connecting the Android Application	21
5.1.	Install the Application	21
5.2.	Enable Bluetooth	21
5.3.	Connect to the ESP32	21
5.4.	Connection Status	21
5.5.	Debug Mode (Viewing Bluetooth Communication)	21
5.5.1.	Enable Debug Mode	22
5.5.2.	Open the Serial Monitor	22
5.5.3.	Debug Output	22
5.5.4.	Troubleshooting Using Debug Mode	23
5.6.	Using Debug Mode to Verify Communication	23
5.6.1.	Step 1: Enable Debug Mode	23
5.6.2.	Step 2: Open the Serial Monitor	23
5.6.3.	Step 3: Move Controls in the App	24
5.7.	Debug Output Types	24
5.7.1.	Joystick Debug (Sticks)	24
5.7.2.	Knob Debug	24
5.7.3.	Switch Debug	25
5.7.4.	Event Debug	25
5.8.	Debugging the Telemetry System	26
5.8.1.	Normal Operation	26
5.8.2.	Numeric Panel Debug Mode	27
5.8.3.	Indicator Debug Mode	28
5.8.4.	Plot Debug Mode	29
5.8.5.	How the Telemetry Debug System Works	29
5.9.	Why Debug Mode is Important	29
5.10.	Customizing Telemetry Labels in the Android Application	30
5.10.1.	Where the Labels Are Defined	30
5.10.2.	Plot Labels	30
5.10.3.	Numeric Panel Labels	31
5.10.4.	Indicator Labels	31
5.10.5.	When the Labels Are Sent	31
5.10.6.	Important Notes	32
6.	Basic Controls	33
6.1.	RC Screen Layout	33

6.2.	Joystick Control	33
6.3.	Switches (Discrete Channels)	34
6.4.	Knobs (Analog Channels)	34
6.5.	Center Buttons	34
6.6.	Indicators and Telemetry	34
6.7.	Summary	35
7.	Customizing Your Project	36
7.1.	Pin Mapping (pins.h) and Project Modes	36
7.1.1.	I ² C bus (reserved / MCP bus)	36
7.1.2.	PWM outputs (6 total)	37
7.1.3.	Motor direction outputs (2)	37
7.1.4.	Analog inputs (ADC1-safe)	37
7.1.5.	Digital inputs (indicators) in direct mode	38
7.1.6.	Switch outputs (direct mode)	38
7.1.7.	Pulse outputs (Android event buttons)	38
7.2.	Adding Sensors (Telemetry Sources)	39
7.2.1.	Analog sensors	39
7.2.2.	I ² C sensors	39
7.3.	Using MCP23017 (GPIO Expansion)	39
7.4.	Practical Checklist When Customizing	40
8.	Example Projects	41
8.1.	RC Car (Differential Drive)	41
8.2.	Camera Control Platform (Pan/Tilt Head)	42
8.3.	Sensor Rover (Telemetry-First Build)	43
8.4.	Custom Robotics Platform (Modular, Expandable)	44
8.5.	Project Planning Checklist (Recommended)	44
9.	Wiring Diagrams	46
9.1.	Basic Robot Wiring	46
9.2.	Servo Control Example	46
9.3.	LED and Buzzer Outputs	47
9.4.	Sensor Connections	47
10.	Troubleshooting	48
10.1.	Before You Start: Quick Checklist	48
10.2.	ESP32 Firmware Does Not Upload	48
10.3.	ESP32 Powers On but Resets Randomly	49
10.4.	Bluetooth Pairing and Connection Problems (Android ↔ ESP32)	49
10.5.	Controls Feel Wrong in the App (Android Side)	50
10.6.	Motors Do Not Move	50
10.7.	Servo Does Not Respond or Jitters	51
10.8.	Telemetry Data Not Appearing in the App	51
10.9.	I2C Devices Not Detected (MCP23017 or Sensors)	51
10.10.	General Debugging Tips (Recommended Workflow)	52
11.	Advanced Features	53
11.1.	Protocol Overview: Commands, Events, and Telemetry	53

11.2. Telemetry System (Panels, Indicators, Plots)	53
11.2.1. Telemetry categories	53
11.2.2. Telemetry update rates (practical meaning)	54
11.2.3. Automatic configuration handshake (labels)	54
11.2.4. Panel resend window (robustness feature)	54
11.3. Telemetry Sources and Debug Injection	54
11.4. Project Modes: ESP32-Only vs MCP23017 Expansion	55
11.5. Using the MCP23017 GPIO Expander (I2C)	55
11.5.1. Key details (from the implementation)	55
11.5.2. Practical applications	55
11.6. I2C Sensor Support (Examples Included)	55
11.7. Analog Input Design for Bluetooth Compatibility	56
11.8. Feature Flags (Build-Time Modular Firmware)	56
11.9. Putting It All Together: Example “Advanced Build”	56
12. Practical Control and Telemetry Examples	57
12.1. Introduction	57
12.2. Control Data from the Android Application	57
12.2.1. RC state packet (rcStatePacket)	57
12.2.2. Event packet (rcEventPacket)	58
12.3. How the ESP32 Receives Packets	58
12.4. Where Control Values are Applied	58
12.5. Example 1: Steering Control Using the Left Joystick	58
12.6. Example 2: Differential Drive Motor Speed Control	59
12.6.1. Direction + speed mapping (implemented behavior)	59
12.6.2. Smooth ramping (advanced behavior)	59
12.7. Example 3: Camera Pan Control	60
12.8. Example 4: Using Knobs to Control LED and Buzzer PWM	60
12.9. Example 5: Controlling Digital Outputs with Switches	60
12.10. Example 6: Handling Button Events (Pulse Outputs)	61
12.10.1. Non-blocking pulses	61
12.11. Telemetry Data Sent to the Android Application	61
12.11.1. Telemetry types and rates (implemented)	62
12.11.2. Config-on-connect	62
12.12. Example 7: Battery Telemetry as an Indicator	62
12.13. Example 8: Telemetry Panels from Analog Inputs	62
12.14. Example 9: Plot Signals (0..255 Fast Graphs)	63
12.15. Example 10: Digital Mask Indicators	63
12.16. Summary	63
13. Control Layer and Hardware Abstraction	64
13.1. Purpose of This Chapter	64
13.2. Layered Architecture (High-Level View)	64
13.3. Control Packets and State Structures	65
13.4. Control Layer (control.cpp)	65
13.4.1. Responsibilities	65
13.4.2. Main loop: controlUpdate()	66
13.4.3. Switch bit mask decoding	66

13.4.4. Event handling: <code>controlHandleEvent(eventId)</code>	66
13.5. Hardware Abstraction Layer (<code>hal_output.cpp</code>)	66
13.5.1. Responsibilities	66
13.5.2. PWM outputs (examples)	67
13.5.3. Motor control: direction + dead-band + ramping	67
13.5.4. Switch outputs: direct GPIO vs MCP23017	67
13.5.5. Pulse outputs (momentary actions)	67
13.6. Pin Mapping (<code>pins.h</code>) and Portability	68
13.7. Summary	68
14. Wiring Diagram Explained (Repository Reference)	69
14.1. Diagram (Included)	69
14.2. Quick Summary	70
14.3. Straight Explanation of Each Group (What it does)	71
14.3.1. I2C (SDA 21, SCL 22)	71
14.3.2. Analog telemetry inputs (NUM1/NUM2/AIN1/AIN2)	71
14.3.3. Digital indicator inputs (IND1–IND4)	71
14.3.4. PWM outputs (PWM LX/LY/RX/RY/KL/KR)	71
14.3.5. Switch outputs (SW1–SW6)	71
14.3.6. Pulse/event outputs	71
14.4. Resume (One Paragraph)	72

Capítulo 1

Introducción

1.1. Qué es el Sistema de Controlador RC con ESP32

El sistema de Controlador RC con ESP32 te permite controlar proyectos electrónicos utilizando un teléfono Android y un microcontrolador ESP32.

La aplicación Android se comunica con el ESP32 mediante Bluetooth. Los comandos de control enviados desde el teléfono llegan al ESP32, que luego actualiza motores, servos, LEDs y otros dispositivos conectados a la placa.

Al mismo tiempo, el ESP32 puede enviar información de vuelta al teléfono, por ejemplo:

- Valores de sensores
- Estados de botones
- Estado del sistema
- Telemetría en tiempo real

Esto crea un sistema de comunicación bidireccional entre la aplicación Android y el ESP32.

1.2. Para Quién es Esta Guía

Esta guía está dirigida a:

- Estudiantes que están aprendiendo electrónica
- Aficionados a la robótica
- Makers que construyen proyectos DIY
- Principiantes que usan la plataforma ESP32

Solo se requiere conocimiento básico del IDE de Arduino y de cableado electrónico sencillo.

1.3. Qué Puedes Construir

Con el sistema de Controlador RC puedes construir muchos tipos de proyectos:

- Autos RC
- Robots
- Sistemas de paneo para cámaras
- Plataformas de monitoreo de sensores
- Dispositivos controlados por Bluetooth

El firmware fue diseñado para ser modular, lo que permite a los usuarios ampliar el sistema con sensores adicionales, motores y dispositivos de control.

Capítulo 2

Resumen de la Arquitectura del Código ESP32

Este capítulo explica la arquitectura interna del repositorio `ESP32_BT_Controller` basándose en el diagrama al final de este capítulo. Describe cómo la aplicación Android se comunica con el firmware del ESP32, cómo se decodifican los comandos de entrada, y cómo esos comandos se convierten en salidas reales de hardware (motores, señales PWM, interruptores), con telemetría y funciones de depuración opcionales.

2.1. Arquitectura de Alto Nivel (Android App ↔ ESP32)

El sistema está construido alrededor de un enlace **Bluetooth SPP (Serial Port Profile)**:

- La **Android App** envía dos tipos de paquetes:
 - **Paquetes de Estado RC** (estado de control continuo: joysticks, perillas, máscara de bits de interruptores).
 - **Paquetes de Evento** (eventos de pulsación de botones que disparan pulsos cortos).
- El **firmware del ESP32** recibe estos paquetes, actualiza estructuras de datos compartidas, y aplica continuamente el estado de control más reciente a las salidas de hardware.
- Opcionalmente, el ESP32 envía **paquetes de telemetría** de vuelta a la aplicación Android (datos de panel/indicador/gráfica y un mensaje de configuración de una sola vez).

2.2. Capa de Inicialización (Diagrama: *Initialization*)

Al inicio, `ESP32_BT_Controller.ino` ejecuta `setup()`, que llama a `systemInit()`. Esta secuencia de arranque prepara las comunicaciones y el hardware.

2.2.1. Inicialización Serial (`serialInit()`)

El firmware inicia el puerto serial USB a 115200 baudios. Esto se usa principalmente para diagnósticos de desarrollo (impresiones de depuración), y no afecta directamente el control por Bluetooth.

2.2.2. Inicialización Bluetooth (`bluetoothInit()`)

Bluetooth SPP se inicializa con:

- `SerialBT.begin("ESP32-BT")`

Esto crea el nombre del dispositivo con el que la aplicación se empareja y se conecta.

2.2.3. Inicialización de Hardware (`hardwareInit()`)

La configuración de hardware está dividida en módulos para mantener el proyecto mantenible:

- `boardInit()` — configuración a nivel de placa (modos GPIO, valores por defecto de la placa).
- `i2cBusInit()` — prepara el bus I²C (si se usa).
- `halOutputsInit()` — prepara los canales de salida PWM (LEDC) y los pines de salida.
- Opcional: `mcpInit()` — inicializa un expensor de E/S MCP23017 si está habilitado.
- Opcional: `i2cSensorsInit()` — inicializa sensores externos en I²C si está habilitado.

2.3. Capa de Ejecución (Diagrama: *Runtime / main loop*)

Después de la inicialización, la función Arduino `loop()` se ejecuta continuamente. En este repositorio:

- `loop()` llama a `systemLoop()` (procesamiento central de control).
- `loop()` también llama a `pulseUpdate()` para salidas momentáneas.

Dentro de `systemLoop()`, el flujo principal de ejecución es:

1. **Recibir y decodificar comandos Bluetooth** (`handleBluetooth()`).
2. **Enviar telemetría cuando corresponda** (`sendTelemetryIfDue()`), solo cuando una aplicación está conectada.
3. **Aplicar salidas de control** (`controlUpdate()`).
4. **Impresión de depuración opcional** (controlada por `debug_config.h`).

2.4. Subsistema de Recepción Bluetooth (Diagrama: *Bluetooth RX (receiver)*)

El módulo receptor Bluetooth (`receiver.cpp`) es responsable de manejar el flujo de bytes crudo. Debido a que los datos Bluetooth pueden llegar fragmentados o consecutivos, el receptor usa un enfoque de **buffer rodante**:

- Leer bytes desde `SerialBT`.
- Buscar cabeceras de paquete válidas.
- Esperar hasta que la longitud completa del paquete esté presente.

- Copiar el paquete a una estructura de paquete compartida.

2.4.1. Paquetes de Entrada

El repositorio define formatos de paquetes fijos en `packets.h`:

Paquete de Estado RC (RcPacket) Una instantánea continua de valores de control (joysticks, perillas, interruptores). Esta es la entrada principal para el control de motor/servo/PWM.

Paquete de Evento (EventPacket) Un paquete corto que contiene un `eventId`. Cuando se recibe, el firmware dispara un manejador de eventos para generar una acción de pulso.

2.5. Modelo de Datos Compartido (Diagrama: *packets.h*)

El archivo `packets.h` es la definición central de todos los formatos de mensajes usados en el sistema. Es importante arquitectónicamente porque actúa como el contrato compartido entre módulos:

- El receptor escribe el estado/evento decodificado más reciente en instancias globales de paquetes.
- La capa de control lee esas instancias de paquetes en cada iteración del bucle.
- La capa de telemetría crea paquetes salientes usando diseños fijos y sumas de verificación.

Esta separación hace que el sistema sea más fácil de extender: añadir un campo de telemetría o un nuevo tipo de paquete se hace de forma centralizada.

2.6. Capa de Control (Diagrama: *Control layer*)

La capa de control (`control.cpp`) convierte el estado RC recibido más recientemente en acciones de hardware:

- Lee valores de joystick/perilla/interruptor del paquete de estado actual.
- Mapea rangos de control a rangos PWM.
- Llama funciones de salida HAL tales como:
 - salida PWM de dirección,
 - velocidad y dirección del motor,
 - PWM de paneo de cámara,
 - brillo de LED,
 - intensidad del buzzer,
 - salidas de interruptores.

2.6.1. Manejo de Eventos

Cuando llega un `EventPacket`, el receptor dispara `controlHandleEvent(eventId)`. Este manejador de eventos mapea un `eventId` a una acción de pulso corta (por ejemplo: pulsar un interruptor de salida específico).

2.7. Capa de Abstracción de Hardware (Diagrama: *HAL outputs*)

El módulo de salida HAL (`hal_output.cpp`) es responsable de las operaciones de hardware de bajo nivel:

- Generación PWM usando LEDC del ESP32 para servos, motores, brillo de LED e intensidad del buzzer.
- Control de dirección del motor usando pines GPIO dedicados de dirección.
- Rampa del motor usando una interrupción de temporizador para suavizar aceleración/desaceleración.
- Salidas de interruptores accionadas ya sea:
 - directamente por pines GPIO del ESP32, o
 - mediante el expensor MCP23017 opcional.

2.8. Expansión GPIO Opcional (Diagrama: *MCP23017 (optional)*)

Si está habilitado, el MCP23017 proporciona líneas extra de E/S sobre I²C. El módulo `mcp_io.cpp` maneja:

- Escritura en registros del MCP para controlar salidas.
- Lectura de entradas desde el Puerto A del MCP (comúnmente usado para indicadores).
- Escritura de salidas en el Puerto B del MCP (comúnmente usado para interruptores/eventos).

2.9. Subsistema de Telemetría (Diagrama: *Telemetry TX y Telemetry sources*)

El subsistema de telemetría (`telemetry.cpp`) envía datos de vuelta a la aplicación Android, pero solo cuando un cliente está conectado (`SerialBT.hasClient()`).

2.9.1. Comportamiento Consciente de la Conexión

- Si no hay un teléfono conectado, la telemetría no hace nada y reinicia su estado de “configuración enviada”.

- Cuando un teléfono se conecta, el ESP32 primero envía un **mensaje de telemetría de configuración** de una sola vez (etiquetas para gráficas/paneles/indicadores) para que la aplicación pueda completar su interfaz.
- Después de eso, envía paquetes de telemetría periódicamente según intervalos de tiempo.

2.9.2. Fuentes de Telemetría

Los valores de telemetría son producidos por funciones fuente separadas (mostradas como *Telemetry sources* en el diagrama), por ejemplo lecturas ADC, máscaras digitales y valores de sensores opcionales. El módulo de telemetría se enfoca en la construcción de paquetes y la planificación, no en decidir cómo se recogen los datos crudos.

2.10. Subsistema de Depuración (Diagrama: *Debug*)

El subsistema de depuración (`debug.cpp`) solo formatea e imprime información a `Serial`. Está controlado por banderas de compilación en `debug_config.h`:

- las impresiones de joystick, perilla, interruptor y evento pueden habilitarse/deshabilitarse.
- la depuración no implementa lógica Bluetooth ni buffering de memoria; está separada intencionalmente.

2.11. Subsistema de Pulsos (Diagrama: *Pulse outputs*)

Algunas acciones del usuario son momentáneas (una pulsación de botón debería crear un pulso corto). Este repositorio maneja ese comportamiento en el subsistema de pulsos:

- `pulseUpdate()` se llama en el `loop()` de Arduino.

Esto mantiene la “temporización de pulsos” separada de la actualización principal de control y evita que las salidas se queden encendidas involuntariamente.

2.12. Resumen: Por qué la Arquitectura está Estructurada de esta Manera

El repositorio está organizado como una tubería de módulos especializados:

- **Receiver** aísla la decodificación del flujo de bytes Bluetooth.
- **Packets** define el contrato de comunicación compartido.
- **Control** mapea entradas de la aplicación en acciones.
- **HAL** realiza escrituras de salida específicas de hardware.
- **Telemetry** maneja la planificación y el reporte de salidas.
- **Debug** proporciona visibilidad sin contaminar el código de lógica.

Esta separación hace que el firmware sea más fácil de mantener y extender (nuevas salidas, nuevos campos de telemetría, nuevos sensores), mientras mantiene estable la comunicación con la aplicación Android.

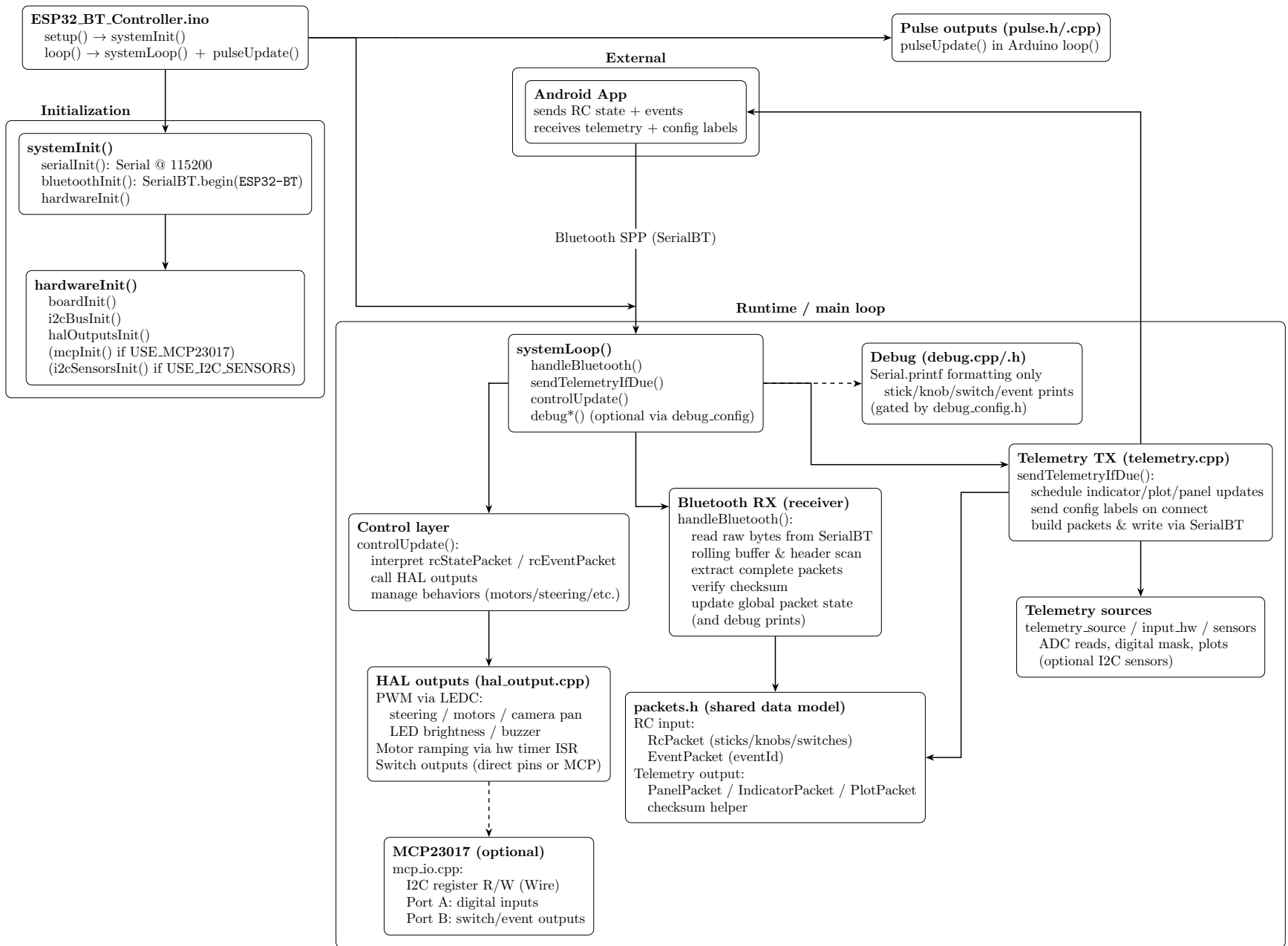


Figura 2.1: Arquitectura de software del ESP32.

Capítulo 3

Configuración de Hardware

Este capítulo describe la configuración de hardware recomendada para el firmware **MICSBO-L/ESP32_BT_Controller**: el ESP32 recibe paquetes de control desde la aplicación Android a través de Bluetooth (SPP), acciona actuadores mediante la capa de abstracción de hardware (HAL) y, de forma opcional, puede leer entradas de sensores para telemetría.

3.1. Placa de Desarrollo ESP32

El ESP32 actúa como el controlador principal. Después de cargar (flashear) el firmware y alimentar la placa, esta anuncia un dispositivo Bluetooth llamado ESP32-BT. Si Android solicita un PIN durante el emparejamiento, el PIN por defecto es 1234. (Perfil Bluetooth: SPP.)¹

Una configuración típica incluye:

- una placa de desarrollo ESP32 con el firmware del repositorio cargado,
- una alimentación estable de 5V (batería externa USB / cargador / o fuente regulada de 5V),
- actuadores conectados (driver de motor, servos, LEDs/zumbador), y
- sensores opcionales (analógicos o I²C) para telemetría.

Modos del proyecto El firmware admite diferentes enfoques de cableado según el modo seleccionado en `user_config.h`:

- `MODE_FULL_RC` (por defecto): utiliza los pines integrados del ESP32 para las salidas.
- `MODE_FULL_RC_MCP`: amplía las salidas digitales usando un expensor GPIO MCP23017.

3.2. Salidas de Control (Lo que Controla la App)

Una vez conectada, la aplicación Android proporciona canales de control que se mapean en la capa de control/HAL del firmware:

¹Consulta la guía de conexión en el `README.md` del repositorio para el nombre exacto del dispositivo, el PIN por defecto y una referencia rápida.

- Stick izquierdo X: dirección (izquierda/derecha)
- Stick izquierdo Y: velocidad del motor izquierdo
- Stick derecho X: paneo de cámara (izquierda/derecha)
- Stick derecho Y: velocidad del motor derecho
- Perilla izquierda: brillo de LED (0 = apagado, máx. = máximo)
- Perilla derecha: intensidad del zumbador (0 = silencio, máx. = máximo)
- Interruptores 1–6: salidas digitales (encendido/apagado)
- Botones de evento: envían un pulso corto a una salida tipo interruptor

Estas asignaciones se resumen en el `README.md` del repositorio y se implementan en el firmware en `control.cpp` y `hal_output.cpp`.

3.3. Conexión del Driver de Motor

Los motores DC **no** deben alimentarse directamente desde los pines del ESP32. Se requiere un driver de motor (por ejemplo, un módulo de doble puente H).

Normalmente el driver necesita:

- **Alimentación de motor (VM):** voltaje de batería adecuado para los motores,
- **Alimentación lógica (VCC):** normalmente 3.3V o 5V según el módulo del driver,
- **GND:** tierra común compartida con el ESP32, y
- **Entradas de control:** PWM y dirección, controladas por GPIO del ESP32 (a través del HAL).

La tierra común es obligatoria Si el driver de motor y el ESP32 no comparten una tierra común, las señales PWM y de dirección serán poco confiables y los motores pueden no responder.

3.4. Conexiones de Servos

Los servomotores pueden controlarse desde pines GPIO del ESP32 con capacidad PWM. El firmware trata algunos canales como salidas PWM que pueden asignarse a la dirección y al paneo de cámara.

Alimentación del servo Aunque la señal PWM proviene del ESP32, la mayoría de servos deben alimentarse desde una línea separada de 5V:

- Servo **V+**: regulador externo de 5V (dimensionado para la corriente de bloqueo del servo)
- Servo **GND:** compartido con GND del ESP32
- Servo **Señal:** pin PWM del ESP32

3.5. Entradas Opcionales y Sensores de Telemetría

El firmware puede enviar telemetría de vuelta a la app Android (paneles numéricos, indicadores y gráficas). El repositorio describe elementos de telemetría como paneles, indicadores (aceleración/batería) y gráficas (voltios/amperios/RPM). Los valores de telemetría pueden provenir de sensores reales (analógicos/I²C) o pueden inyectarse para pruebas mediante los modos de depuración de telemetría en `debug_config.h`.

Interfaces típicas de sensores:

- **Entradas analógicas (ADC):** para monitor de batería, potenciómetros, sensores de temperatura, etc.
- **I²C:** para sensores externos o expansión de E/S (MCP23017 en modo MCP)

3.6. Fuente de Alimentación y Seguridad

Un sistema de alimentación confiable es esencial para una operación Bluetooth estable y un control correcto de motores/servos.

3.6.1. Esquema de alimentación recomendado

- **ESP32:** alimentado por USB de 5V o una línea regulada de 5V.
- **Motores:** alimentados desde una batería/línea de motor a través del driver (VM).
- **Servos (si se usan):** alimentados por un regulador separado de 5V capaz de entregar picos de corriente altos.

3.6.2. Tierra (GND)

Conecta siempre todas las tierras en común:

- GND del ESP32
- GND del driver de motor
- GND de la alimentación de servos
- GND de los sensores

3.6.3. Notas prácticas

- Mantén el cableado de potencia de motores físicamente separado del cableado de señales del ESP32 cuando sea posible.
- Agrega capacitores de desacoplo cerca de drivers de motor / rieles de servos para reducir reinicios por caídas de tensión.
- Si el ESP32 se reinicia cuando arrancan los motores, la causa suele ser caída de tensión o ruido: mejora el regulador, añade capacitancia y verifica la topología del cableado de tierra.

Capítulo 4

Carga del Firmware

Este capítulo explica cómo instalar el entorno de desarrollo y cargar el firmware en el microcontrolador ESP32. El firmware habilita la comunicación Bluetooth con la aplicación móvil y proporciona la lógica de control para motores, servos, sensores y telemetría.

El procedimiento utiliza el IDE de Arduino, que ofrece un entorno sencillo para compilar y cargar firmware en placas de desarrollo ESP32.

4.1. Instalar el IDE de Arduino

Descarga e instala el IDE de Arduino desde el sitio web oficial:

`https://www.arduino.cc/en/software`

Se admiten tanto Arduino IDE 1.8.x como Arduino IDE 2.x, pero se recomienda Arduino IDE 2.x porque ofrece mejor rendimiento y una interfaz de usuario moderna.

Después de instalar el software, abre el IDE de Arduino para continuar con la configuración del ESP32.

4.2. Instalar el Soporte de Placas ESP32

El IDE de Arduino requiere el paquete de placas ESP32 para poder compilar firmware para microcontroladores ESP32.

Abre el IDE de Arduino y navega a:

`File -> Preferences`

Ubica el campo **Additional Boards Manager URLs** y agrega la siguiente dirección:

`https://dl.espressif.com/dl/package\_esp32\_index.json`

Presiona **OK** para guardar la configuración.

A continuación, instala las definiciones de la placa ESP32:

`Tools -> Board -> Boards Manager`

Busca:

ESP32 by Espressif Systems

Instala el paquete de la placa.

Para obtener la mejor compatibilidad con este proyecto, se recomienda la siguiente versión:

ESP32 Core Version 2.0.x

Esta versión proporciona soporte Bluetooth estable y funciona bien con la arquitectura del firmware utilizada en este proyecto.

4.3. Descargar el Firmware

Descarga el código fuente del proyecto desde el repositorio de GitHub:

https://github.com/MICSBOL/ESP32_BT_Controller

Puedes descargar el proyecto haciendo clic en el botón **Code** y seleccionando **Download ZIP**, o clonando el repositorio usando Git.

Después de descargar el proyecto, extrae los archivos en una carpeta de tu computadora.

4.4. Abrir el Proyecto del Firmware

Abre el IDE de Arduino y selecciona:

File -> Open

Navega hasta la carpeta que contiene el firmware y abre el archivo principal del proyecto:

ESP32_BT_Controller.ino

Cuando el archivo se abre, el IDE de Arduino cargará automáticamente todos los archivos fuente adicionales que usa el firmware, incluyendo los componentes modulares responsables de la lógica de control, comunicación, telemetría e interfaces de hardware.

4.5. Seleccionar la Placa ESP32

Configura el tipo de placa usado para la compilación.

Navega a:

Tools -> Board

Selecciona:

ESP32 Dev Module

Esta configuración es compatible con la mayoría de placas de desarrollo ESP32 disponibles en el mercado.

4.6. Configurar Parámetros de la Placa

Para un funcionamiento confiable, se recomienda usar la siguiente configuración de placa:

- Board: ESP32 Dev Module
- Upload Speed: 921600
- CPU Frequency: 240 MHz
- Flash Frequency: 80 MHz
- Flash Mode: QIO
- Flash Size: 4MB
- Partition Scheme: Default 4MB
- Core Debug Level: None
- PSRAM: Disabled

Estos ajustes aseguran una compilación estable y una carga de firmware confiable para la mayoría de placas ESP32.

4.7. Conectar la Placa ESP32

Conecta la placa de desarrollo ESP32 a tu computadora usando un cable USB.

Después de conectar la placa, selecciona el puerto serie utilizado por el dispositivo:

Tools -> Port

Nombres típicos de puertos incluyen:

COM3
COM4
/dev/ttyUSB0
/dev/ttyACM0

Selecciona el puerto correspondiente a tu dispositivo ESP32.

4.8. Compilar el Firmware

Antes de cargar el firmware, se recomienda verificar que el código compila correctamente.

Haz clic en el botón **Verify** en el IDE de Arduino.

El IDE compilará todos los archivos fuente usados por el proyecto. Si la compilación termina correctamente, la consola mostrará un mensaje indicando el uso de memoria del programa.

4.9. Cargar el Firmware

Haz clic en el botón **Upload** para transferir el firmware compilado al ESP32.

Durante el proceso de carga, el IDE compilará el firmware y lo enviará al microcontrolador a través de la conexión USB.

Algunas placas ESP32 requieren presionar el botón **BOOT** cuando comienza la carga. Si la carga falla, mantén presionado el botón BOOT mientras el firmware se está cargando.

Cuando la carga finaliza, el IDE mostrará un mensaje indicando que el ESP32 se está reiniciando.

4.10. Verificar el Funcionamiento del Firmware

Abre el Monitor Serie para confirmar que el firmware está ejecutándose.

Navega a:

Tools -> Serial Monitor

Configura la velocidad de comunicación serie en:

115200

Cuando el ESP32 arranca, deberías ver mensajes de estado impresos en la consola indicando que el controlador Bluetooth se ha iniciado y está listo para aceptar conexiones desde la aplicación móvil.

4.11. Siguiente Paso

Después de cargar el firmware correctamente, el siguiente paso es conectar la aplicación móvil al ESP32 y comenzar a controlar el dispositivo.

El proceso de conexión se describe en el Capítulo 5.

Capítulo 5

Conexión de la Aplicación Android

5.1. Instalar la Aplicación

Instala la aplicación Android de RC Controller en tu teléfono usando el método de distribución proporcionado por tu proyecto (APK, enlace de tienda o despliegue interno).

5.2. Activar Bluetooth

Activa Bluetooth en tu teléfono antes de abrir la aplicación. En versiones recientes de Android, la app puede solicitar permisos adicionales (por ejemplo, “Dispositivos cercanos” o permisos relacionados con Ubicación) para poder escanear dispositivos Bluetooth. Concede los permisos solicitados para que la lista de dispositivos se muestre correctamente.

5.3. Conectarse al ESP32

Enciende el ESP32 y abre la aplicación RC Controller. En la lista de dispositivos Bluetooth disponibles, selecciona el controlador ESP32.

Una vez establecida la conexión, la interfaz de control se activa y la aplicación empieza a enviar datos de control al ESP32 (joysticks, perillas, interruptores y eventos de botones).

5.4. Estado de Conexión

La aplicación muestra el estado de conexión para confirmar que el enlace serie Bluetooth está activo.

Del lado del firmware, un cliente conectado se detecta con `SerialBT.hasClient()` (utilizado por el módulo de telemetría). Si el teléfono se desconecta, el firmware vuelve al estado de “espera” y la aplicación debe reconectar antes de que el control y la telemetría se reanuden.

5.5. Modo Debug (Ver la Comunicación Bluetooth)

El firmware incluye un sistema de depuración opcional que imprime datos del controlador ya decodificados en el Monitor Serie USB. Esta es la forma más rápida de confirmar que:

- el teléfono está conectado al dispositivo ESP32 correcto,
- los paquetes se están recibiendo y decodificando correctamente, y
- los valores de sticks/perillas/interruptores cambian cuando mueves los controles en la app.

5.5.1. Activar el Modo Debug

Abre el archivo:

```
debug_config.h
```

Activa la depuración estableciendo:

```
#define DEBUG_ENABLED 1
```

Desactiva la depuración estableciendo:

```
#define DEBUG_ENABLED 0
```

Cuando DEBUG_ENABLED está activo, el repositorio también habilita categorías individuales de depuración:

```
#define DBG_STICKS    1
#define DBG_KNOBS    1
#define DBG_SWITCHES 1
#define DBG_EVENTS    1
```

Después de cambiar cualquier ajuste de depuración, compila y carga el firmware nuevamente en el ESP32.

5.5.2. Abrir el Monitor Serie

Abre el Monitor Serie del IDE de Arduino:

```
Tools -> Serial Monitor
```

Configura la velocidad (baud rate) en:

```
115200
```

Reinicia el ESP32 para ver los mensajes de arranque.

5.5.3. Salida de Depuración

Cuando el modo debug está habilitado, el ESP32 imprimirá mensajes de diagnóstico en el Monitor Serie.

Mensajes típicos pueden incluir:

```
[DEBUG] Bluetooth initialized
[DEBUG] Waiting for client connection
[DEBUG] Client connected
```

Cuando se reciben comandos desde la aplicación Android, el ESP32 puede mostrar mensajes similares a:

```
[DEBUG] Received command: BTN_A
[DEBUG] Joystick X: 120
[DEBUG] Joystick Y: 85
```

Estos mensajes confirman que la comunicación Bluetooth entre el teléfono y el ESP32 está funcionando correctamente.

5.5.4. Solución de Problemas Usando el Modo Debug

Si la aplicación Android se conecta pero el robot no responde:

- Verifica que aparezcan mensajes de depuración al presionar botones en la aplicación.
- Si no aparece ningún mensaje, es posible que el ESP32 no esté recibiendo datos Bluetooth.
- Comprueba que en la aplicación se haya seleccionado el dispositivo ESP32 correcto.
- Verifica que el firmware se haya cargado correctamente.

Si aparecen mensajes de depuración pero el hardware no responde, el problema probablemente está relacionado con el cableado o con la configuración de pines.

5.6. Usar el Modo Debug para Verificar la Comunicación

El firmware incluye un sistema de depuración integrado que permite ver los datos que se reciben desde la aplicación Android.

Esto es extremadamente útil para comprobar que la conexión Bluetooth funciona correctamente y que el ESP32 está recibiendo comandos de control.

Todos los mensajes de depuración se muestran en el Monitor Serie del IDE de Arduino.

5.6.1. Paso 1: Activar el Modo Debug

Abre el archivo:

```
debug_config.h
```

Ubica la siguiente línea:

```
#define DEBUG_ENABLED 1
```

Ajusta el valor a:

- 1 para habilitar la salida de depuración
- 0 para deshabilitar la salida de depuración

Cuando la depuración está habilitada, el firmware imprimirá los datos de control recibidos desde la aplicación Android.

Vuelve a cargar el firmware después de cualquier cambio.

5.6.2. Paso 2: Abrir el Monitor Serie

En el IDE de Arduino abre:

Tools -> Serial Monitor

Configura la velocidad a:

115200

Reinicia el ESP32 o presiona el botón de reset.

El Monitor Serie empezará a mostrar mensajes de depuración.

5.6.3. Paso 3: Mover Controles en la App

Después de conectar la aplicación Android al ESP32, mueve los controles en pantalla:

- Mueve los joysticks
- Gira las perillas
- Activa o desactiva los interruptores
- Presiona botones

El ESP32 imprimirá los datos recibidos en el Monitor Serie.

5.7. Tipos de Salida de Depuración

El sistema de depuración puede imprimir diferentes categorías de información según la configuración en `debug_config.h`.

5.7.1. Depuración de Joystick (Sticks)

Si la siguiente opción está habilitada:

```
#define DBG_STICKS 1
```

El ESP32 imprimirá las posiciones de los joysticks cuando cambien.

Ejemplo:

```
L Stick X: 512    L Stick Y: 510
L Stick X: 650    L Stick Y: 480

R Stick X: 300    R Stick Y: 700
```

Estos valores representan la posición del joystick izquierdo y derecho en la aplicación Android.

Rango típico:

- Mínimo: alrededor de 0
- Centro: alrededor de 512
- Máximo: alrededor de 1023

Si los números cambian cuando mueves el joystick, la comunicación está funcionando correctamente.

5.7.2. Depuración de Perillas (Knobs)

Si la siguiente opción está habilitada:

```
#define DBG_KNOBS 1
```

El ESP32 imprimirá los valores de las perillas.

Ejemplo:

```
Left Knob: 340
Left Knob: 512
Left Knob: 890
```

```
Right Knob: 120
Right Knob: 780
```

Estos valores corresponden a la rotación de las perillas en la aplicación Android.

Las perillas pueden usarse para controlar parámetros como:

- Velocidad del motor
- Ángulo del servo
- Intensidad de luz
- Inclinación/paneo de cámara

5.7.3. Depuración de Interruptores (Switches)

Si la siguiente opción está habilitada:

```
#define DBG_SWITCHES 1
```

El ESP32 mostrará el estado de los interruptores.

Ejemplo:

```
Switch Byte: 0x05
S1 = ON
S2 = OFF
S3 = ON
S4 = OFF
S5 = OFF
S6 = OFF
```

Cada interruptor corresponde a un botón tipo toggle en la aplicación Android.

Estos interruptores se usan comúnmente para:

- Encender y apagar luces
- Activar relés
- Cambiar modos de conducción
- Habilitar funciones autónomas

5.7.4. Depuración de Eventos

Si la siguiente opción está habilitada:

```
#define DBG_EVENTS 1
```

El ESP32 imprimirá un mensaje cuando llegue un paquete de evento.

Ejemplo:

```
--- EVENT PACKET (PRESS) ---  
Event ID: 0x03  
Checksum: 0x7A  
-----
```

Los eventos se disparan cuando el usuario presiona un botón en la aplicación Android.

Usos típicos:

- Activar una función especial
- Reiniciar un sistema
- Disparar un sonido o animación
- Lanzar una acción en el robot

5.8. Depuración del Sistema de Telemetría

El firmware incluye un sistema de telemetría que permite al ESP32 enviar información en tiempo real de vuelta a la aplicación Android.

La telemetría puede incluir:

- Valores numéricos
- Indicadores
- Gráficas en tiempo real

Para desarrollo y pruebas, el firmware proporciona un sistema de depuración de telemetría que permite simular valores mediante el Monitor Serie del IDE de Arduino.

Este sistema se configura en el archivo:

`debug_config.h`

Dentro de este archivo puedes seleccionar el modo de depuración de telemetría:

```
#define DBG_NONE          0
#define DBG_PANEL         1
#define DBG_INDICATOR     2
#define DBG_PLOT          3

#define TELEMETRY_DEBUG_MODE DBG_INDICATOR
```

Solo debe estar activo un modo de depuración a la vez.

5.8.1. Operación Normal

```
#define TELEMETRY_DEBUG_MODE DBG_NONE
```

En este modo el ESP32 obtiene valores de telemetría desde sensores reales de hardware.

Ejemplos:

- sensores analógicos
- medición de voltaje de batería
- entradas digitales

Este modo debe usarse cuando el robot o proyecto esté completamente armado.

5.8.2. Modo Debug de Panel Numérico

```
#define TELEMETRY_DEBUG_MODE DBG_PANEL
```

Este modo permite probar los paneles numéricos de telemetría sin conectar sensores.

Cuando este modo está habilitado, los valores ingresados en el Monitor Serie se interpretan y se envían a la aplicación Android como telemetría de panel numérico.

Formato de entrada en el Monitor Serie

Abre el Monitor Serie del IDE de Arduino y envía valores con el siguiente formato:

```
LEFT_VALUE,RIGHT_VALUE
```

Ambos valores pueden incluir parte decimal.

Ejemplos:

```
12.3,45.8
120.0,350.5
5,10
```

Conversión de valores

Los valores ingresados en el Monitor Serie se convierten internamente al formato de telemetría usado por el firmware.

Cada valor se multiplica por 10 y se convierte a entero antes de transmitirse a la aplicación Android.

Ejemplos:

- 12.3 se convierte en 123
- 45.8 se convierte en 458
- 120.0 se convierte en 1200

Del lado Android, el valor entero recibido se divide por 10 para mostrar el valor decimal correcto en el panel numérico.

Rango válido

El rango permitido para cada valor numérico es:

- 0.0 a 6553.5

Este rango corresponde al valor máximo transmisible usando un entero sin signo de 16 bits con una cifra decimal de precisión.

Ejemplo

Si se ingresa el siguiente valor en el Monitor Serie:

123.4,56.7

El firmware transmitirá:

1234,567

La aplicación Android mostrará:

123.4 56.7

en los paneles numéricos izquierdo y derecho respectivamente.

Este modo es útil para verificar la transmisión de telemetría y la interfaz Android antes de conectar sensores reales.

5.8.3. Modo Debug de Indicadores

```
#define TELEMETRY_DEBUG_MODE DBG_INDICATOR
```

Este modo permite probar widgets de indicadores.

Envía valores por el Monitor Serie usando este formato:

```
ANALOG_VALUE, BATTERY_LEVEL, DIGITAL_MASK
```

Ejemplo:

45,80,5

Rangos:

- Valor analógico: 0 a 100

- Nivel de batería: 0 a 100
- Máscara digital: 0 a 255

La máscara digital representa el estado de indicadores digitales usando flags por bits.

Ejemplos:

1
2
4
8

Cada valor activa un indicador diferente.

Se pueden activar múltiples indicadores combinando valores.

Ejemplo:

3

Representación binaria:

00000011

Esto activa los dos primeros indicadores.

5.8.4. Modo Debug de Gráficas (Plot)

```
#define TELEMETRY_DEBUG_MODE DBG_PLOT
```

Este modo simula tres señales de telemetría que se muestran como gráficas en tiempo real en la aplicación Android.

Envía valores usando el Monitor Serie:

VALUE1,VALUE2,VALUE3

Ejemplo:

10,40,80

Rango válido:

- 0 a 255

Estos valores aparecerán en las gráficas de telemetría de la aplicación.

5.8.5. Cómo Funciona el Sistema de Depuración de Telemetría

Los valores de depuración son leídos por el módulo:

telemetry_source.cpp

Este módulo reemplaza valores de sensores reales por valores recibidos vía interfaz serie cuando hay un modo debug activo.

Los paquetes de telemetría se crean y transmiten mediante:

telemetry.cpp

Este módulo arma paquetes de telemetría y los envía a la aplicación Android usando Bluetooth. Esta arquitectura permite probar la interfaz de telemetría sin conectar sensores físicos.

5.9. Por Qué el Modo Debug es Importante

El modo debug permite verificar rápidamente que:

- La aplicación Android está conectada
- Se están recibiendo datos por Bluetooth
- Los valores de control cambian correctamente

Si los valores aparecen en el Monitor Serie pero el hardware no responde, el problema probablemente está en el cableado de hardware o en la configuración del firmware, y no en la conexión Bluetooth.

5.10. Personalizar Etiquetas de Telemetría en la Aplicación Android

La aplicación Android muestra automáticamente etiquetas para valores de telemetría como paneles numéricos, indicadores y gráficas.

Estas etiquetas se definen en el firmware del ESP32 y se envían a la aplicación cuando se establece la conexión Bluetooth.

Esto permite personalizar la interfaz sin modificar la aplicación Android.

5.10.1. Dónde se Definen las Etiquetas

Las etiquetas de telemetría se definen dentro del archivo:

`telemetry.cpp`

Dentro de la función encargada de enviar la configuración de telemetría, encontrarás estas variables:

```
const char* plotNames[] = { "Volts", "Amps", "RPMs" };
const char* panelNames[] = { "Left", "Right" };
const char* indicatorNames[] = { "Throttle", "Battery" };
```

Estos arreglos definen las etiquetas que aparecen en la pantalla RC de la aplicación Android.

5.10.2. Etiquetas de Gráficas (Plot)

El arreglo `plotNames` define los nombres de las gráficas en tiempo real.

Ejemplo:

```
const char* plotNames[] = { "Volts", "Amps", "RPMs" };
```

En la aplicación Android esto creará tres gráficas con las etiquetas:

- Volts
- Amps
- RPMs

Puedes renombrarlas para que coincidan con tus sensores.

Ejemplo para un robot tipo auto:

```
const char* plotNames[] = { "Speed", "Motor Temp", "Battery Current" };
```

5.10.3. Etiquetas de Panel Numérico

El arreglo `panelNames` define las etiquetas de los paneles numéricos grandes.

Ejemplo:

```
const char* panelNames[] = { "Left", "Right" };
```

Usos típicos:

- Velocidad de motor
- Distancia
- RPM
- Temperatura

Ejemplo de modificación:

```
const char* panelNames[] = { "Motor RPM", "Battery Voltage" };
```

Nota: se recomienda usar un máximo de 12 caracteres.

5.10.4. Etiquetas de Indicadores

El arreglo `indicatorNames` define las etiquetas de los widgets de indicadores.

Ejemplo:

```
const char* indicatorNames[] = { "Throttle", "Battery" };
```

Posibles usos:

- Posición del acelerador
- Nivel de batería
- Temperatura
- Intensidad de señal

Ejemplo de modificación:

```
const char* indicatorNames[] = { "Engine Load", "Battery Level" };
```

5.10.5. Cuándo se Envían las Etiquetas

Las etiquetas de telemetría se transmiten solo una vez después de que la aplicación Android se conecta al ESP32.

El firmware envía un paquete de configuración de telemetría que incluye:

- etiquetas de gráficas
- etiquetas de paneles numéricos
- etiquetas de indicadores

Después de recibir este paquete, la aplicación Android actualiza automáticamente la interfaz de la pantalla RC.

5.10.6. Notas Importantes

- El número de etiquetas debe coincidir con el número de variables de telemetría.
- Después de modificar las etiquetas, debes volver a cargar el firmware.
- La aplicación Android actualizará las etiquetas automáticamente tras reconectar.

Capítulo 6

Controles Básicos

Este capítulo describe los principales controles del operador disponibles en la *Pantalla RC*. La interfaz está organizada en el lado izquierdo y el lado derecho del controlador, además de un área central de telemetría, lo que permite conducir el robot mientras se monitorea el estado del sistema en tiempo real.

6.1. Distribución de la Pantalla RC

La Pantalla RC se compone de tres regiones principales:

- **Lado Izquierdo del Controlador:** controles primarios (dependen del modo del joystick), interruptores, perilla y un widget de estado en la parte superior.
- **Pantalla Central:** una gráfica de telemetría utilizada para visualizar datos en vivo.
- **Lado Derecho del Controlador:** controles secundarios (dependen del modo del joystick), interruptores, perilla y un widget de estado en la parte superior.

Un panel de fondo proporciona contraste para mejorar la legibilidad, y la pantalla está pensada para usarse en **orientación horizontal** (landscape) para un espaciado más ergonómico.

6.2. Control con Joystick

Cada lado del controlador incluye un joystick virtual. El comportamiento del joystick depende del **modo de stick** seleccionado (configurado en los ajustes del usuario):

- **Modo resorte (Spring):** el joystick vuelve al centro al soltarlo (útil para entradas momentáneas como dirección o ajuste fino de velocidad).
- **Modo fijo (Hold):** el joystick mantiene su posición al soltarlo (útil para aceleración sostenida o consignas).
- **Modos direccionales:** algunas configuraciones pueden enfatizar el control horizontal vs. vertical según los requisitos del robot.

Usos típicos del joystick:

- Movimiento hacia adelante/atrás (aceleración)

- Giro izquierda/derecha (dirección)
- Paneo/inclinación de cámara (cuando se asigna a un gimbal)
- Cualquier canal de control continuo requerido por el robot (por ejemplo, control de consignas)

6.3. Interruptores (Canales Discretos)

Tanto el lado izquierdo como el lado derecho proporcionan **interruptores tipo toggle** para canales de control discretos. Los interruptores están pensados para acciones que son *encendido/apagado* o que seleccionan entre un número pequeño de estados, por ejemplo:

- Habilitar/deshabilitar motores
- Seleccionar modos de conducción (p. ej., lento/normal/boost)
- Encender/apagar dispositivos auxiliares (luces, bomba, ventilador, etc.)

6.4. Perillas (Canales Analógicos)

Cada lado también incluye una **perilla giratoria** para ajuste analógico. Las perillas son útiles para afinar parámetros mientras se conduce:

- Sensibilidad o ajuste fino (trim) de la dirección
- Límite de velocidad
- Zoom o enfoque de cámara (si es compatible)
- Ganancias PID u otros parámetros de ajuste (configuraciones avanzadas)

6.5. Botones Centrales

La Pantalla RC incluye **botones pulsadores** cerca del área central (arriba y abajo por lado). Están pensados para acciones rápidas que deben ejecutarse inmediatamente, por ejemplo:

- Bocina / zumbador
- Reset / confirmación (acknowledge)
- Tomar foto / iniciar grabación (carga útil de cámara)
- Disparar una rutina automatizada

6.6. Indicadores y Telemetría

La interfaz proporciona varias formas de retroalimentación para que el operador pueda monitorear el sistema mientras lo controla:

- **Indicador analógico (widget superior izquierdo):** muestra un valor numérico con un título (p. ej., RPM, temperatura, intensidad de señal).

- **Estado de batería (widget superior derecho):** muestra el nivel de batería con una etiqueta de título.
- **Indicadores LED:** un conjunto de LEDs para señales rápidas de estado (p. ej., estado armado, advertencias, estado de enlace).
- **Indicadores de panel:** paneles por lado que muestran un valor, un estado encendido/apagado, un color y un título (útiles para canales de telemetría con nombre).
- **Gráfica central:** una gráfica en vivo que muestra una o varias series de telemetría a lo largo del tiempo.

Estos indicadores ayudan a confirmar la recepción de comandos y permiten detectar problemas como batería baja, pérdida de enlace o advertencias de sensores de forma temprana.

6.7. Resumen

En general:

- Usa los **joysticks** para el control continuo de movimiento.
- Usa **interruptores y botones** para acciones discretas y cambios de modo.
- Usa las **perillas** para ajustes de precisión.
- Usa **widgets de telemetría y gráficas** para monitorear el estado del sistema durante la operación.

Capítulo 7

Personalización de tu Proyecto

Este capítulo explica la forma recomendada de ampliar el firmware en MICSBOL/ESP32_BT_Controller. El repositorio está organizado de manera que:

- **El mapeo de pines está centralizado** en `pins.h` y depende del modo de compilación seleccionado mediante `PROJECT_MODE`.
- **Las lecturas de hardware** (ADC/GPIO/I²C) se recolectan en módulos dedicados como `input_hw.*` (lecturas ADC/digitales) y `i2c_sensors.*`.
- **Los paquetes de telemetría** se construyen y se transmiten en `telemetry.cpp`, pero los *valores* provienen de un módulo proveedor separado, `telemetry_source.*`.
- **La expansión opcional de GPIO** (MCP23017) se gestiona mediante `mcp_io.*` y el mapeo dependiente del modo en `pins.h`.

7.1. Mapeo de Pines (`pins.h`) y Modos de Proyecto

El repositorio define dos modos de cableado en `pins.h`:

- **Modo directo ESP32** (`PROJECT_MODE == FULL_RC_MODE`): la mayoría de E/S usa pines GPIO del ESP32.
- **Modo expansión MCP** (`PROJECT_MODE == FULL_RC_MODE_MCP`): la E/S digital se amplía usando un MCP23017 por I²C, mientras que las salidas PWM se mantienen en el ESP32.

7.1.1. Bus I²C (reservado / bus MCP)

En ambos modos, `pins.h` reserva el bus I²C del ESP32 en:

- `PIN_I2C_SDA = 21`
- `PIN_I2C_SCL = 22`

En modo MCP, estos pines se usan para la comunicación ESP32 ↔ MCP23017 (y también pueden compartirse con otros sensores I²C si el bus está diseñado correctamente).

7.1.2. Salidas PWM (6 en total)

En ambos modos, `pins.h` define seis salidas PWM en el ESP32:

- `PIN_PWM_STICK_LX` = 18
- `PIN_PWM_STICK_LY` = 19
- `PIN_PWM_STICK_RX` = 23
- `PIN_PWM_STICK_RY` = 25
- `PIN_PWM_KNOB_L` = 26
- `PIN_PWM_KNOB_R` = 27

Estos canales suelen usarse para servos, entradas PWM de drivers de motor u otras salidas de tipo analógico.

7.1.3. Salidas de dirección de motor (2)

Ambos modos definen dos salidas de dirección pensadas para drivers de motor:

- `PIN_MD_LEFT` = 12 (nota en el repositorio: *mantener en LOW al arranque*)
- `PIN_MD_RIGHT` = 15 (nota en el repositorio: *mantener en LOW al arranque*)

Nota de seguridad al arranque Los comentarios en `pins.h` advierten que estos pines deben mantenerse en LOW durante el arranque. Al personalizar, evita resistencias pull-up externas o circuitos que lleven estos pines a HIGH durante el reset.

7.1.4. Entradas analógicas (ADC1 seguras)

El repositorio usa explícitamente pines de ADC1 para entradas analógicas (recomendado cuando se usa Bluetooth). Sin embargo, los números de pin cambian según el modo:

Modo directo ESP32 (`FULL_RC_MODE`)

- `PIN_NUMERIC1` = 34
- `PIN_NUMERIC2` = 35
- `PIN_ANALOG_IND1` = 36
- `PIN_ANALOG_IND2` = 39

Modo expansión MCP (`FULL_RC_MODE_MCP`)

- `PIN_NUMERIC1` = 32
- `PIN_NUMERIC2` = 33
- `PIN_ANALOG_IND1` = 34
- `PIN_ANALOG_IND2` = 35

Implicación práctica Si cambias de modo de proyecto, **debes** revisar de nuevo el cableado de los sensores analógicos, porque los pines ADC para numéricos/indicadores cambian entre modos.

7.1.5. Entradas digitales (indicadores) en modo directo

En modo directo ESP32, `pins.h` define cuatro entradas digitales para indicadores:

- `PIN_IND1 = 4`
- `PIN_IND2 = 5`
- `PIN_IND3 = 2`
- `PIN_IND4 = 0` (nota del repositorio: *cuidado: evitar pull-up al arranque*)

Advertencia de arranque en GPIO0 Como `PIN_IND4` usa GPIO0 en modo directo, evita un cableado que fuerce GPIO0 a HIGH/LOW durante el reset de una forma que ponga al ESP32 en un modo de arranque incorrecto.

7.1.6. Salidas de interruptores (modo directo)

En modo directo, el repositorio define salidas de interruptores en pines del ESP32:

- `PIN_SW1 = 16`
- `PIN_SW2 = 17`
- `PIN_SW3 = 13`
- `PIN_SW4 = 14`

Importante En el estado actual de `pins.h`, solo se definen `PIN_SW1` hasta `PIN_SW4` en modo directo, aunque otras partes del sistema se refieren a “Switches 1–6” de forma conceptual. Si tu proyecto necesita SW5/SW6 en modo directo, debes asignar pines GPIO adicionales en `pins.h` y actualizar la capa de salidas en consecuencia.

7.1.7. Salidas de pulso (botones de evento Android)

Las salidas de pulso están pensadas para acciones momentáneas disparadas por botones de evento en Android.

Modo directo ESP32

- `PIN_PULSE1 = 32`
- `PIN_PULSE2 = 33`

Modo expansión MCP En modo MCP, `pins.h` asigna cuatro salidas de pulso en GPIO del ESP32:

- `PIN_PULSE1 = 4`

- `PIN_PULSE2 = 5`
- `PIN_PULSE3 = 13`
- `PIN_PULSE4 = 14`

7.2. Agregar Sensores (Fuentes de Telemetría)

La forma más limpia de agregar sensores es alimentar sus mediciones en los canales de telemetría existentes. El API del proveedor se declara en `telemetry_source.h`:

- **Paneles:** `getPanelLeft()` y `getPanelRight()` devuelven valores de 16 bits.
- **Indicadores:** `getIndicatorAnalog()`, `getIndicatorBattery()` y `getIndicatorDigitalMask()` devuelven valores de 8 bits.
- **Gráficas (plots):** `getPlot1()`, `getPlot2()` y `getPlot3()` devuelven valores de 8 bits.

7.2.1. Sensores analógicos

Flujo de trabajo típico:

1. Elige un pin ADC desde la sección del modo correcto en `pins.h`.
2. Implementa el escalado/conversión en `input_hw.cpp` (consulta funciones como `hwReadPlot1()`, `hwReadIndicatorAnalog()`, etc., declaradas en `input_hw.h`).
3. Devuelve el valor procesado en `telemetry_source.cpp` mediante `getPlotX()/getIndicator...()`

7.2.2. Sensores I²C

El repositorio ofrece helpers I²C en `i2c_sensors.*` (por ejemplo, `i2cReadTemp()` y `i2cReadIMU(...)` en `i2c_sensors.h`). Para agregar otro dispositivo:

1. Conecta SDA/SCL a `PIN_I2C_SDA/PIN_I2C_SCL`.
2. Agrega una función de lectura en `i2c_sensors.cpp/.h`.
3. Publica el valor a través de `telemetry_source.cpp`.

7.3. Uso de MCP23017 (Expansión GPIO)

En modo MCP, `pins.h` documenta el mapeo digital previsto:

- **GPA0–GPA7** → 8 indicadores digitales (ENTRADAS)
- **GPB0–GPB5** → 6 salidas de interruptores
- **GPB6–GPB7** → 2 salidas libres para que las uses

La interfaz del MCP en `mcp_io.h` ofrece:

- `mcpInit()`
- `mcpReadDigitalInputs()` (lee el puerto A)

- `mcpWriteOutput(...)` y `mcpWritePortB(...)` (escribe el puerto B)

7.4. Lista Práctica al Personalizar

- **Comienza siempre revisando `pins.h` para el modo seleccionado:** los pines analógicos y de pulso cambian entre modos.
- **Ten cuidado con pines sensibles al arranque:** GPIO0 y GPIO12/15 tienen advertencias explícitas relacionadas con el arranque en el repositorio.
- **Si necesitas más E/S digital, prefiere el modo MCP:** amplía de forma limpia indicadores y salidas de interruptores mediante I²C.
- **Cambia una cosa a la vez:** actualiza el cableado, luego actualiza `pins.h`/funciones de lectura, y después prueba telemetría y salidas.
- **Recomendación al flashear:** carga el firmware con el ESP32 **totalmente desconectado del hardware externo** (motores, servos, relés, sensores y especialmente cualquier cableado a pines de arranque/strapping). Esto evita fallos de carga por ruido eléctrico, caídas de tensión o por forzar niveles lógicos incorrectos en pines críticos durante el arranque.
- **Si falla la carga:** primero desconecta todo lo externo, prueba con otro cable USB/u otro puerto, y si tu placa lo requiere mantén presionado el botón **BOOT** al iniciar la carga. Reconecta el hardware externo **solo después** de confirmar que el firmware arranca correctamente.

Capítulo 8

Proyectos de Ejemplo

Este capítulo presenta proyectos de ejemplo prácticos y de la vida real que combinan las dos partes del sistema:

- **EmitterApp (Android)**: proporciona la interfaz del operador (dos joysticks, interruptores, perillas, botones de evento) además de widgets de telemetría en vivo (paneles, indicadores LED, indicadores analógicos/de batería y gráficas).
- **ESP32_BT_Controller (ESP32, C/C++)**: se ejecuta en el hardware del robot/controlador, recibe comandos RC por Bluetooth (SPP) y envía configuración + telemetría de vuelta a la aplicación.

El objetivo de cada ejemplo es mostrar (1) cómo se mapean los controles al hardware real y (2) qué telemetría es útil durante la operación.

8.1. Auto RC (Tracción Diferencial)

Una construcción común en el mundo real es un pequeño auto/rover RC de **tracción diferencial de dos ruedas** (dos motores DC + driver de motor). Este es el proyecto más simple que se beneficia de inmediato de la interfaz de dos sticks.

Hardware típico

- Placa de desarrollo ESP32
- Driver de motor de doble puente H (p. ej., TB6612FNG, DRV8833, L298N)
- 2 motores DC + chasis + batería
- Opcional: sensor de voltaje de batería (divisor resistivo al ADC)

Mapeo de controles (práctico)

- **Stick izquierdo Y**: aceleración (adelante/atrás).
- **Stick izquierdo X**: dirección (girar izquierda/derecha) o “mezcla arcade” (aceleración + dirección combinadas en velocidades de motor izquierdo/derecho).

- **Switch 1:** armar/desarmar motores (seguridad).
- **Perilla:** limitador de velocidad máxima (útil para manejo en interiores o entrenamiento).
- **Botón de evento:** pulso de bocina/zumbador (retroalimentación audible corta).

Telemetría que importa en la vida real

- **Indicador de batería:** evita sobredescargar el paquete.
- **Paneles:** muestran el comando de motor izquierdo/derecho o la velocidad de rueda medida (si hay encoders).
- **Gráfica:** voltaje de batería a lo largo del tiempo (la caída bajo carga muestra cuándo estás cerca de agotarla).

Caso de uso real

Práctica de navegación en interiores, laboratorios básicos de enseñanza de robótica o un auto de demostración para taller donde los operadores pueden *sentir* la diferencia entre modos de control (spring vs hold) mientras observan los efectos de batería y carga en la gráfica.

8.2. Plataforma de Control de Cámara (Cabezal Pan/Tilt)

Una plataforma pan/tilt es un proyecto realista usado para inspección, transmisión o experimentos de visión en robótica. También es un buen ejemplo de mezclar **control continuo** y **ajuste fino**.

Hardware típico

- Placa de desarrollo ESP32
- 2 servos (pan + tilt) o un gimbal pequeño
- Cámara (cámara de acción, cámara FPV pequeña, soporte para teléfono, etc.)
- Opcional: luz LED para iluminación

Mapeo de controles (práctico)

- **Stick derecho X:** pan de cámara.
- **Stick derecho Y:** tilt de cámara.
- **Perilla:** velocidad/suavizado del servo (movimiento cinematográfico lento vs seguimiento rápido).
- **Switch:** alternar luz LED (encendido/apagado).
- **Botón de evento:** acción “centrar cámara” (envía un evento corto para reiniciar ángulos).

Telemetría que importa en la vida real

- **Indicador analógico:** ángulo actual de pan o “ángulo objetivo de pan”.
- **Panel:** ángulo de tilt o un modo con nombre (p. ej., *TRACK* vs *LOCK*).
- **Indicadores LED:** muestran si la plataforma está centrada, siguiendo o en estado de advertencia.

Caso de uso real

Inspección en el hogar (mirar debajo de muebles, techos), ajuste de visión en robótica (apuntar la cámara mientras se ajusta la iluminación), o una demostración en vivo en un taller donde una persona conduce y otra controla la cámara.

8.3. Rover con Sensores (Construcción “Telemetry-First”)

Un rover con sensores enfatiza la **medición y la retroalimentación** más que la velocidad. El ESP32 lee sensores y transmite telemetría al dispositivo Android, donde el operador ve paneles y gráficas de series temporales.

Hardware típico

- Placa de desarrollo ESP32
- Sensor de distancia (ultrasónico o ToF)
- Sensor ambiental (temperatura/humedad/presión) por I2C
- Opcional: expensor GPIO I2C MCP23017 (si necesitas más E/S)
- Chasis de rover (2WD o 4WD)

Mapeo de controles (práctico)

- **Stick izquierdo:** conducción lenta (precisión).
- **Switches:** alternar modos de sensores (p. ej., muestreo rápido/lento, registro on/off).
- **Perilla:** tasa de muestreo o umbral de alerta (p. ej., límite de advertencia de distancia).

Telemetría que importa en la vida real

- **Panel izquierdo:** distancia (cm) al obstáculo.
- **Panel derecho:** temperatura (°C) o humedad (%).
- **Gráficas:** tendencias de distancia y/o voltaje (útiles para diagnosticar ruido, pérdidas de lectura del sensor o problemas de alimentación).
- **Indicador de batería:** crítico para experimentos largos.

Caso de uso real

Mapeo ambiental (mover el rover por una habitación para ver gradientes de temperatura), experimentos de distancia a obstáculos para prototipos de conducción autónoma o demostraciones de operación basada en telemetría donde la gráfica revela lo que ocurre incluso cuando el robot está fuera de la vista.

8.4. Plataforma Robótica Personalizada (Modular, Expandible)

Un proyecto más avanzado del mundo real es un **robot modular** que combina locomoción, actuadores y múltiples dispositivos accesorios. Aquí es donde usar ambos repositorios juntos se vuelve especialmente valioso: el lado Android ofrece una interfaz rica para el operador, y el lado ESP32 puede adaptarse (mapeo de pines, modos, expansión I2C, fuentes de telemetría).

Hardware típico

- Base de tracción (orugas o ruedas)
- Actuador (brazo con servo, actuador lineal, pinza)
- Luces + zumbador para retroalimentación
- Varios sensores (monitoreo de batería, encoders, distancia, IMU)
- Opcional: MCP23017 para switches/salidas adicionales

Ejemplo de mapeo de controles (práctico)

- **Stick izquierdo:** conducción.
- **Stick derecho:** control del brazo (rotación + elevación) o control de cámara.
- **Switches:** modos (habilitar brazo, modo precisión, luces).
- **Perillas:** ajuste fino (límite de velocidad, fuerza de agarre, trim del servo).
- **Botones de evento:** acciones discretas (abrir/cerrar pinza, reiniciar orientación, paro de emergencia).

Telemetría que importa en la vida real

- **Widget de batería:** seguridad del operador (detenerse antes de brownouts).
- **Paneles:** señales con nombre como *ARM CURR*, *IMU YAW*, *GRIP STATE*.
- **LEDs:** estado rápido (armado, advertencia, falla, enlace ok).
- **Gráfica:** una o más señales a lo largo del tiempo (p. ej., consumo de corriente vs comando de motor para detectar atascos).

Caso de uso real

Proyectos maker y clubes de robótica: una plataforma base puede reutilizarse para diferentes eventos (sumo bot, robot de inspección, demostración de divulgación) cambiando la configuración del firmware (pines/funciones) y reutilizando la misma pantalla de control Android.

8.5. Lista de Verificación para Planificación del Proyecto (Recomendada)

Antes de construir, decide:

- **Controles:** qué debe hacer cada eje del joystick, switch, perilla y botón.
- **Seguridad:** se recomienda fuertemente un switch de armar/desarmar y un evento de paro de emergencia.
- **Telemetría:** elige 2–4 valores clave que hagan la operación más segura (batería, distancia, corriente de motor, temperatura).
- **Expansión:** si necesitas dispositivos I2C (sensores, MCP23017) para escalar E/S.

Capítulo 9

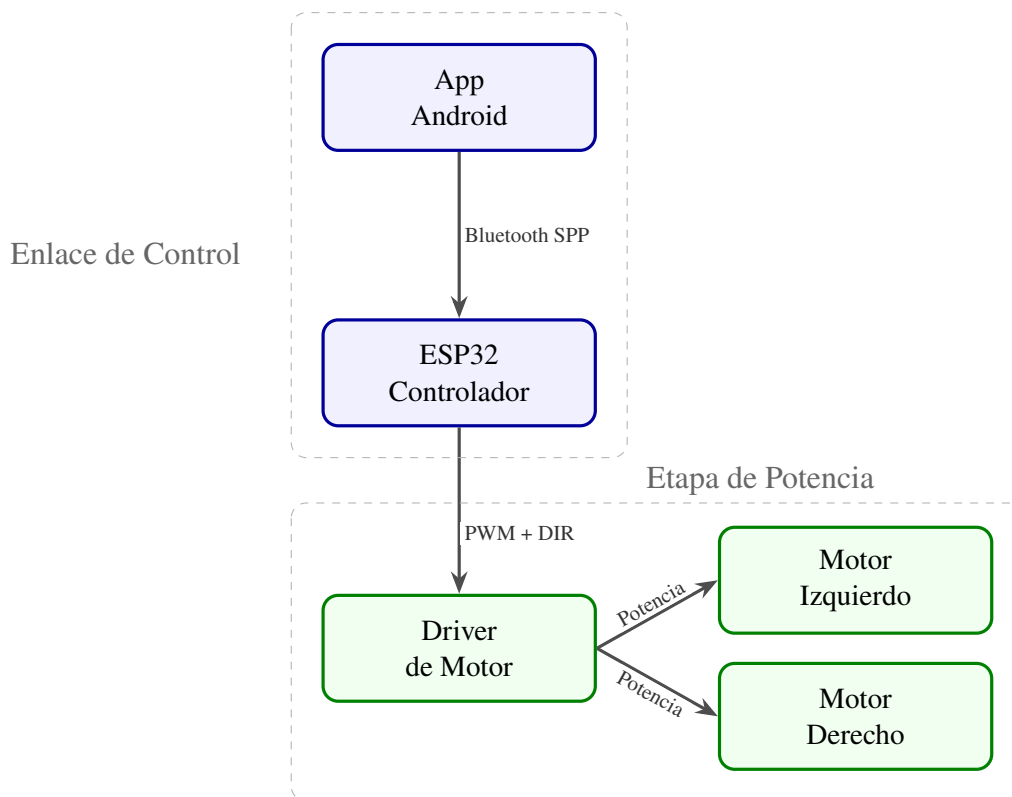
Diagramas de Cableado

Este capítulo muestra ejemplos básicos de cableado para proyectos comunes usando el firmware del Controlador RC con ESP32.

Estos diagramas están simplificados para que los principiantes puedan entender rápidamente cómo conectar el hardware.

9.1. Cableado Básico de Robot

El proyecto más común es un robot de tracción diferencial simple usando dos motores.



9.2. Ejemplo de Control de Servo

Los servos pueden conectarse directamente a pines PWM del ESP32.

- Señal del servo → pin GPIO del ESP32
- Alimentación del servo → fuente de 5V
- Tierra del servo → tierra común

Ejemplos de uso:

- Control de dirección
- Control de paneo de cámara
- Articulaciones de brazo robótico

9.3. Salidas para LED y Zumbador

Las salidas digitales pueden controlar LEDs o zumbadores.

Conexiones de ejemplo:

- Pin GPIO → LED (con resistencia)
- Pin GPIO → módulo zumbador

Estas salidas pueden controlarse usando botones o interruptores en la aplicación Android.

9.4. Conexiones de Sensores

Los sensores pueden conectarse usando:

- Entradas analógicas
- Entradas digitales
- Interfaz I2C

El firmware permite transmitir datos de sensores a la aplicación Android mediante telemetría.

Capítulo 10

Solución de Problemas

Si tu proyecto no funciona como se espera, este capítulo te ayuda a identificar problemas comunes en **ambos lados del sistema**:

- **Android (EmitterApp)**: controles de la interfaz, emparejamiento/conexión Bluetooth y si se están procesando comandos/telemetría.
- **ESP32 (ESP32_BT_Controller)**: carga del firmware, cableado de pines, integridad de alimentación, enlace Bluetooth SPP y generación de telemetría.

10.1. Antes de Empezar: Lista Rápida

- Confirma que el ESP32 está alimentado y estable (sin brownouts/resets).
- Confirma que tu dispositivo Android está emparejado con el nombre Bluetooth correcto (por defecto es ESP32-BT).
- Mantén el teléfono a pocos metros para las pruebas iniciales.
- Asegúrate de que **todas las tierras sean comunes** (ESP32, driver de motor, sensores, fuente externa de 5V/alimentación de servos).
- Si estás usando pines sensibles al arranque (por ejemplo, GPIO0), desconecta temporalmente los circuitos externos durante el flasheo.

10.2. El Firmware del ESP32 No Se Carga

Si el firmware no se carga correctamente:

- Verifica que la placa/target correcto esté seleccionado en tu IDE/herramientas.
- Confirma que el puerto serie correcto esté seleccionado.
- Prueba con otro cable USB (muchos cables “solo carga” causan fallos de carga).
- Mantén presionado **BOOT** durante la carga si tu placa requiere entrada manual al bootloader.

- Desconecta el cableado externo de los **pines de arranque (boot-strap)** durante el flasheo (los más comunes son GPIO0 y GPIO2).

Notas sobre pines de arranque (importante)

La configuración de pines del firmware usa GPIO0 como entrada digital de indicador (PIN_IND4 = GPIO0) en FULL_RC_MODE. GPIO0 es un pin de arranque: resistencias pull-up/pull-down externas pueden impedir el arranque o el flasheo. Si el flasheo falla, desconecta cualquier cosa conectada a GPIO0, flashea de nuevo y luego vuelve a conectar.

10.3. El ESP32 Enciende pero Se Reinicia Aleatoriamente

Los reinicios aleatorios suelen estar relacionados con la alimentación:

- Los motores/servos pueden bajar el voltaje de forma repentina. Usa un regulador dedicado para el ESP32 y una fuente separada para motores/servos.
- Asegúrate de que la fuente del ESP32 pueda entregar corriente pico (ráfagas de Wi-Fi/Bluetooth pueden causar caídas).
- Agrega capacitancia de reserva cerca de drivers de motor/riel de servos (p. ej., 470–1000 μ F) y un capacitor cerámico más pequeño cerca del ESP32.
- Revisa si los reguladores de las placas de desarrollo se sobrecalientan cuando se alimentan servos desde el mismo riel de 5V (no recomendado).

10.4. Problemas de Emparejamiento y Conexión Bluetooth (Android ↔ ESP32)

Síntoma: El dispositivo no aparece en el escaneo

- Confirma que el ESP32 está alimentado y ejecutándose (LED/indicadores de actividad si están disponibles).
- Asegúrate de que el teléfono esté cerca del ESP32.
- Apaga y vuelve a encender el ESP32.

Síntoma: El emparejamiento pide PIN o falla

El PIN por defecto normalmente es 1234.

- Elimina (“olvida”) el dispositivo en la configuración Bluetooth de Android y empareja de nuevo.
- Intenta emparejar primero en la configuración del sistema Android y luego conéctate desde la app.

Síntoma: La app dice “Connected” pero los controles no hacen nada

- Verifica que te conectaste al dispositivo correcto (si hay varios ESP32-BT cerca).
- Confirma que el ESP32 está ejecutando el build de firmware esperado (no un sketch viejo).
- Del lado del ESP32, usa logging serial (si está habilitado) para confirmar que llegan paquetes de comandos.

Síntoma: Se conecta y luego se desconecta repetidamente

- Esto suele ser un problema de **estabilidad de alimentación** en el ESP32 (ver Sección 10.3).
- Reduce la carga de motores/servos durante las pruebas iniciales (desconecta motores, prueba primero con LEDs).
- Mantén el teléfono cerca y evita carcasas metálicas que bloqueen Bluetooth.

10.5. Los Controles Se Sienten Mal en la App (Lado Android)

El joystick vuelve al centro vs se queda fijo

EmitterApp soporta diferentes modos de joystick (por ejemplo, spring vs hold). Si el robot se comporta como si la aceleración “no se quedara” o la dirección estuviera “pegada”:

- Revisa los modos de stick seleccionados en la configuración de la app.
- Para aceleración, muchos usuarios prefieren el comportamiento **hold**; para dirección, muchos prefieren **spring**.

Los botones/interruptores aparecen pero no afectan el hardware

- Confirma que el firmware del ESP32 mapea esos canales a pines/salidas reales en el modo de proyecto elegido.
- Si usas FULL_RC_MODE_MCP, verifica el cableado del MCP23017 y la configuración de la dirección I2C (dirección por defecto 0x20).

10.6. Los Motores No Se Mueven

Verificaciones eléctricas

- Revisa la alimentación del driver de motor y la polaridad.
- Confirma que la **tierra lógica** del driver esté conectada a la tierra del ESP32.
- Confirma que las salidas del driver estén conectadas a los terminales del motor (invierte cables si la dirección está al revés).

Verificación de señales

- Confirma que los pines PWM coincidan con tu cableado. En el mapeo de pines por defecto, los canales PWM incluyen: GPIO18/19/23/25/26/27.
- Confirma que los pines de dirección estén conectados correctamente: PIN_MD_LEFT = GPIO12 y PIN_MD_RIGHT = GPIO15 (ambos deben mantenerse en low al arranque).
- Si tu driver necesita dos pines de dirección por motor (IN1/IN2), debes adaptar el cableado/el firmware en consecuencia.

Consejo práctico de depuración

Prueba las salidas de motor con un simple “LED + resistencia” en el pin PWM primero (deberías ver cambios de brillo con el movimiento del stick), y luego conecta el driver de motor.

10.7. El Servo No Responde o Tiembla

- Asegúrate de que el servo tenga una fuente de 5V adecuada (**no** alimentes servos medianos/grandes desde la placa del ESP32).
- Confirma que la tierra del servo esté unida a la tierra del ESP32.
- Verifica que la señal de control esté cableada a un pin de salida PWM configurado (por ejemplo, GPIO23 o GPIO25).
- Si el servo tiembla, agrega una mejor fuente de alimentación y reduce el ruido eléctrico (cables más cortos, capacitores, rieles de alimentación separados).

10.8. Los Datos de Telemetría No Aparecen en la App

Síntoma: Conectado, pero los widgets de telemetría no se actualizan

- Espera unos segundos después de conectar: el ESP32 envía la configuración a la app en la primera conexión.
- Verifica que el sensor esté cableado al tipo de entrada correcto:
 - Telemetría analógica: usa pines ADC1 (p. ej., GPIO34/35/36/39 en FULL_RC_MODE).
 - Indicadores digitales: GPIO4/5/2/0 (nota: precaución de arranque en GPIO0).
 - Sensores I2C: SDA GPIO21, SCL GPIO22.
- Si estás usando FULL_RC_MODE_MCP, confirma que el MCP23017 se detecte y esté cableado correctamente.

Síntoma: La telemetría está presente pero los valores se ven mal

- Revisa el escalado del sensor (divisores de voltaje, rango del ADC, unidades).
- Reduce el ruido: acorta cables analógicos, agrega filtrado (filtro RC) y evita tender líneas analógicas junto a cables de motores.

10.9. Dispositivos I2C No Detectados (MCP23017 o Sensores)

- Verifica que SDA esté en GPIO21 y SCL en GPIO22 (según pins.h).
- Confirma que el dispositivo use lógica de 3.3V (o usa conversión de nivel).
- Revisa resistencias pull-up en SDA/SCL (muchas placas incluyen; demasiadas pull-ups en paralelo también pueden causar problemas).
- Confirma la dirección I2C (por defecto del MCP23017 en el repositorio es 0x20).

10.10. Consejos Generales de Depuración (Flujo Recomendado)

1. **Empieza por el enlace:** verifica primero el emparejamiento Bluetooth y una conexión estable.
2. **Prueba salidas sin motores:** LEDs en pines PWM/de switch confirman que la ruta de control funciona.
3. **Agrega un subsistema a la vez:** motores, luego servos, luego sensores, luego expansión I2C.
4. **Usa logging:**
 - ESP32: salida debug por serial (cuando está habilitada en el firmware).
 - Android: revisa los indicadores para confirmar el estado de conexión y la telemetría entrante.
5. **Trata la alimentación como un componente de primera clase:** la mayoría de fallos intermitentes provienen de subvoltaje o fuentes ruidosas.

Capítulo 11

Funciones Avanzadas

El firmware **ESP32_BT_Controller** está diseñado como una plataforma modular de RC + telemetría. Más allá del control básico por joysticks, incluye un protocolo de comunicación liviano, modos de proyecto configurables, un sistema de telemetría estructurado y expansión I2C opcional.

Este capítulo destaca las funciones avanzadas más útiles y explica cómo funcionan en la práctica.

11.1. Resumen del Protocolo: Comandos, Eventos y Telemetría

La comunicación se basa en paquetes (ver `packets.h`). El firmware usa:

- **Paquetes RC State:** estado de control continuo (sticks, perillas, interruptores).
- **Paquetes de Evento:** comandos cortos de una sola vez (pulsaciones de botones).
- **Paquetes de Telemetría:** retroalimentación desde el ESP32 hacia la app Android (paneles, indicadores, gráficas y etiquetas de configuración).

Todos los paquetes usan:

- Encabezados fijos (p. ej., `0xCC` seguido por un byte de tipo de paquete)
- Estructuras empaquetadas para un parseo predecible
- Un checksum aditivo simple (liviano y rápido para sistemas embebidos)

11.2. Sistema de Telemetría (Paneles, Indicadores, Gráficas)

La telemetría es una de las partes más potentes de este firmware. Permite que el ESP32 envíe valores en tiempo real de vuelta a la app Android para que el operador pueda *ver lo que el robot está haciendo* mientras lo conduce.

11.2.1. Categorías de telemetría

El firmware genera múltiples flujos de telemetría (implementados en `telemetry.cpp`):

- **Paneles** (valores numéricos izquierda/derecha): lecturas compactas como distancia, velocidad, temperatura o cualquier número derivado de un sensor.
- **Indicadores:** un conjunto pequeño de “widgets de estado” que incluye:
 - un valor analógico (0–100)
 - un nivel de batería (0–100)
 - una máscara digital (campo de bits) para múltiples indicadores on/off
- **Gráficas (plots):** series de actualización rápida para gráficos en el tiempo (las etiquetas por defecto incluyen Volts, Amps y RPMs).

11.2.2. Frecuencias de actualización de telemetría (significado práctico)

La telemetría se envía de forma periódica (constantes de tiempo en `telemetry.cpp`):

- **Gráficas (plots):** cada 50 ms ($\approx 20\text{ Hz}$) — gráficos suaves para señales que cambian rápido.
- **Indicadores:** cada 500 ms ($\approx 2\text{ Hz}$) — valores de UI estables para batería/estado.

11.2.3. Handshake automático de configuración (etiquetas)

Cuando el teléfono se conecta, el ESP32 envía un **paquete de telemetría de configuración una vez por conexión** (`sendConfigTelemetry()`). Este paquete proporciona a la app etiquetas legibles para:

- nombres de series de gráficas (p. ej., Volts, Amps, RPMs)
- nombres de paneles (p. ej., Left, Right)
- nombres de indicadores (p. ej., Throttle, Battery)

Esta es una característica importante de usabilidad: la app Android puede poblar las etiquetas automáticamente sin codificarlas de forma fija para cada construcción de robot.

11.2.4. Ventana de reenvío de panel (característica de robustez)

Los valores de panel usan un pequeño mecanismo de reenvío (“ventana de reintento”) de modo que cuando una lectura de panel cambia, se retransmite repetidamente por un periodo corto (ver `resendWindow` y `resendInterval` en `telemetry.cpp`). Esto mejora la probabilidad de que el teléfono reciba la lectura actualizada incluso si se pierde un paquete.

11.3. Fuentes de Telemetría e Inyección de Debug

La telemetría está dividida intencionalmente en dos capas:

- **`telemetry.cpp`** es *solo escritura*: construye paquetes y los envía.
- **`telemetry_source.cpp`** decide *de dónde vienen los datos*: lecturas de hardware o valores de depuración.

En operación normal, los valores de panel/indicador/gráfica se leen desde el hardware mediante funciones de `input_hw.h` (p. ej., entradas numéricas y lecturas ADC). En modos debug, los valores pueden inyectarse por serial escribiendo valores simples separados por comas. Esto permite verificar el comportamiento de la UI Android y la decodificación de paquetes *sin* sensores reales conectados.

Por qué esto importa: Puedes desarrollar primero la UI + pantallas de telemetría y luego integrar el hardware, reduciendo el tiempo de iteración.

11.4. Modos de Proyecto: Solo ESP32 vs Expansión MCP23017

El firmware puede compilarse en diferentes modos de proyecto (configurados en `user_config.h`):

- **MODE_FULL_RC:** control RC completo usando GPIO del ESP32 directamente.
- **MODE_FULL_RC_MCP:** control RC completo usando un MCP23017 por I2C para E/S digital expandida.

Seleccionar el modo correcto es una “función avanzada” porque permite que una sola base de código escale desde robots pequeños hasta construcciones más grandes con muchos interruptores e indicadores.

11.5. Uso del Expansor GPIO MCP23017 (I2C)

El MCP23017 agrega 16 pines de E/S digital por I2C y es compatible con el firmware (`mcp_io.cpp/.h`).

11.5.1. Detalles clave (desde la implementación)

- **Dirección I2C:** `0x20`
- **Puerto A (GPIOA):** configurado como **INPUT** (usado para leer indicadores digitales)
- **Puerto B (GPIOB):** configurado como **OUTPUT** (usado para salidas de interruptores y eventos)
- Las salidas se rastrean por software (`portBState`) para que los pines individuales puedan alternarse de forma confiable.

11.5.2. Aplicaciones prácticas

- Una caja de control con muchos interruptores (armar, luces, modos, accesorios)
- Un robot con muchas entradas de estado (finales de carrera, líneas de falla, sensores de parachoques)
- Arreglos complejos de indicadores LED (sin consumir GPIO del ESP32)

11.6. Soporte de Sensores I2C (Ejemplos Incluidos)

El repositorio incluye ejemplos de lectura de sensores I2C en `i2c_sensors.cpp`:

- Un sensor de temperatura en la dirección `0x48`
- Un ejemplo de IMU en la dirección `0x68` (lecturas de acelerómetro, p. ej., registros estilo MPU6050)

Estos ejemplos actúan como plantillas: puedes agregar pasos de inicialización y registros adicionales para soportar tu módulo de sensor exacto.

11.7. Diseño de Entrada Analógica para Compatibilidad con Bluetooth

Una elección de diseño avanzada, sutil pero importante, es el uso de entradas **solo ADC1** para lecturas analógicas en la configuración de pines por defecto. En ESP32, ADC2 suele ser problemático cuando Wi-Fi/Bluetooth está activo, por lo que el mapeo del repositorio mantiene la telemetría analógica en pines con capacidad ADC1 (ver `pins.h`).

Implicación práctica: Si agregas sensores analógicos, prefiere los pines ADC1 configurados (u otros canales ADC1) para evitar lecturas intermitentes o inválidas mientras Bluetooth está conectado.

11.8. Feature Flags (Firmware Modular en Tiempo de Compilación)

Las banderas de características en tiempo de compilación se definen en `feature_config.h`. Controlan si los subsistemas se compilan/habilitan (salidas PWM, salidas de interruptores, salidas de eventos, telemetría, funciones de sensores I2C, etc.).

Nota: En el snapshot actual del repositorio, `USE_TELEMETRY` y `USE_I2C_SENSOR` se definen como `0` en ambos modos de proyecto, aunque el código de implementación de telemetría existe en el repositorio. Esto sugiere que la telemetría puede habilitarse/deshabilitarse por configuración según las necesidades del proyecto (p. ej., para reducir ancho de banda o uso de CPU).

11.9. Uniendo Todo: Ejemplo de “Construcción Avanzada”

Un proyecto avanzado realista combina estas características:

- **Control RC:** dos sticks + perillas para conducción y control de cámara/brazo
- **MCP23017:** muchos interruptores e indicadores usando expansión por I2C
- **Telemetría:** gráficas de batería + corriente, además de paneles para temperatura y distancia

Esto crea una experiencia robusta para el operador: el control se mantiene responsivo, mientras que la telemetría ofrece conciencia situacional y alerta temprana de caída de batería, atascos de motor o fallas de sensores.

Capítulo 12

Ejemplos Prácticos de Control y Telemetría

12.1. Introducción

Este capítulo proporciona ejemplos prácticos que demuestran cómo interactúan la aplicación Android y el firmware **MICSBOL/ESP32_BT_Controller** en un robot real o un sistema embebido.

Del lado Android se transmite continuamente un **estado de control** (sticks, perillas e interruptores), y también se transmiten **eventos de una sola vez** cuando el usuario presiona botones.

Del lado del ESP32:

- decodifica los paquetes entrantes por Bluetooth Serial (SPP),
- aplica los valores de control recibidos a salidas de hardware reales (PWM, pines de dirección, interruptores),
- y opcionalmente envía telemetría de vuelta a la app Android (paneles, indicadores, gráficas).

12.2. Datos de Control desde la Aplicación Android

12.2.1. Paquete de estado RC (rcStatePacket)

El firmware almacena el estado de control más reciente en la estructura definida en `packets.h`:

```
rcStatePacket
```

Contiene los siguientes campos:

- `leftStickX`, `leftStickY`
- `rightStickX`, `rightStickY`
- `leftKnob`, `rightKnob`
- `switches` (máscara de bits)

En el código del repositorio, los canales de sticks y perillas se tratan como valores de 12 bits y se mapean en el rango **0..4095** (ver `control.cpp`, `hal_output.cpp`).

12.2.2. Paquete de evento (`rcEventPacket`)

Los botones se transmiten como pequeños paquetes de evento, y el ID del evento se pasa a:

```
controlHandleEvent(byte eventId)
```

Los eventos son útiles para acciones que deben ser *momentáneas* (p. ej., disparar, reset, bocina) en lugar de un valor analógico continuo.

12.3. Cómo el ESP32 Recibe Paquetes

Los bytes Bluetooth entrantes se procesan en:

```
receiver.cpp -> handleBluetooth()
```

El receptor mantiene un buffer circular y busca encabezados de paquetes. En la implementación actual del firmware:

- Encabezado RC State: `0xAA 0x55`
- Encabezado Event: `0xBB 0x66`

Una vez decodificado un evento, `controlHandleEvent()` se llama inmediatamente.

12.4. Dónde se Aplican los Valores de Control

El bucle principal de control se implementa en:

```
control.cpp -> controlUpdate()
```

Esta función lee valores de `rcStatePacket` y los envía a la capa de abstracción de hardware (`hal_output.cpp`):

- PWM del servo de dirección
- comandos de motor izquierdo/derecho
- PWM de paneo de cámara
- brillo de LED e intensidad de zumbador (PWM)
- salidas de interruptores (máscara de bits decodificada en salidas por canal)

12.5. Ejemplo 1: Control de Dirección Usando el Joystick Izquierdo

El firmware mapea el eje horizontal del joystick izquierdo a un valor PWM tipo servo:

```
rcStatePacket.data.leftStickX
```

En `control.cpp`, la dirección se genera con:

```
static uint32_t mapServo(uint16_t v) {
    return map(v, 0, 4095, 205, 410);
}

uint32_t steerPWM = mapServo(rcStatePacket.data.leftStickX);
halSetSteering(steerPWM);
```

Interpretación práctica

- El joystick Android produce un valor de 0 a 4095.
- El ESP32 lo mapea a un rango PWM más pequeño (205..410) adecuado para su configuración PWM.

Esto se usa comúnmente para:

- dirección de auto RC
- dirección de ruedas delanteras de robot
- un timón accionado por servo

12.6. Ejemplo 2: Control de Velocidad de Motores en Tracción Diferencial

El firmware usa los ejes verticales de los joysticks como canales de motor izquierdo/derecho:

```
rcStatePacket.data.leftStickY
rcStatePacket.data.rightStickY
```

En `control.cpp`:

```
uint32_t motorL = rcStatePacket.data.leftStickY;
uint32_t motorR = rcStatePacket.data.rightStickY;

halSetMotorLeft(motorL);
halSetMotorRight(motorR);
```

12.6.1. Mapeo de dirección + velocidad (comportamiento implementado)

En `hal_output.cpp`, cada valor de motor se interpreta con una banda muerta (dead-band):

- **0..2000**: pin de dirección LOW (reversa) y PWM mapeado a 0..4095
- **2001..2094**: banda muerta, objetivo del motor se establece en 0
- **2095..4095**: pin de dirección HIGH (adelante) y PWM mapeado a 0..4095

Este diseño reduce el jitter cuando el joystick está cerca del centro.

12.6.2. Rampa suave (comportamiento avanzado)

En lugar de escribir PWM instantáneamente, el PWM del motor hace una rampa hacia un valor objetivo. Una interrupción por timer actualiza las salidas de motor cada 20 ms, moviendo la velocidad por un paso fijo (ver `updateMotors()` en `hal_output.cpp`). Esto mejora la manejabilidad y reduce picos de corriente.

12.7. Ejemplo 3: Control de Paneo de Cámara

El eje horizontal del joystick derecho se usa para paneo de cámara:

```
uint32_t panPWM = mapServo(rcStatePacket.data.rightStickX);  
halSetCameraPan(panPWM);
```

Usos típicos en la vida real:

- cámara de rover de inspección
- posicionamiento de cámara FPV
- montaje de sensor tipo torreta

12.8. Ejemplo 4: Usar Perillas para Controlar PWM de LED y Zumbador

La app Android proporciona dos perillas, transmitidas como valores 0..4095:

```
rcStatePacket.data.leftKnob  
rcStatePacket.data.rightKnob
```

En `control.cpp`, estas se mapean directamente a salidas PWM mediante el HAL:

```
halSetLed(rcStatePacket.data.leftKnob);  
halSetBuzzer(rcStatePacket.data.rightKnob);
```

Usos prácticos

- brillo de LED (faros, underglow, luz de trabajo)
- volumen / intensidad del zumbador (alertas, pitidos de confirmación)
- reutilizar perillas para escalado de límite de motor o trim de servo (modificación personalizada del firmware)

12.9. Ejemplo 5: Controlar Salidas Digitales con Interruptores

Los interruptores se transmiten como una máscara de bits en un solo byte:

```
byte sw = rcStatePacket.data.switches;
```

En `control.cpp`, el firmware itera a través de los índices de switch:

```

for (int i = 0; i < 6; i++) {
    halSetSwitch(i, (sw >> i) & 1);
}

```

Diferencia importante por modo

La cantidad de switches soportados depende del modo de proyecto (ver `hal_output.cpp`):

- **FULL_RC_MODE:** solo los índices 0..3 se escriben a pines GPIO (4 salidas de switch).
- **FULL_RC_MODE_MCP:** los índices 0..5 se escriben usando salidas del MCP23017 (6 salidas de switch).

Usos en el mundo real:

- switch de habilitar/armar (seguridad)
- faros / baliza / underglow
- salidas de relé (bomba, solenoide, alimentación de accesorio)
- selección de modo (manual vs asistido)

12.10. Ejemplo 6: Manejo de Eventos de Botón (Salidas de Pulso)

Los eventos se manejan mediante:

```
controlHandleEvent(byte eventId)
```

En la implementación del repositorio, los eventos llaman a `halPulseSwitch(index)`. El mapeo de índices difiere según el modo de proyecto. Por ejemplo, en `FULL_RC_MODE`:

```

case 0x01: halPulseSwitch(0); break;
case 0x02: halPulseSwitch(1); break;
case 0x03: halPulseSwitch(0); break;
case 0x04: halPulseSwitch(1); break;

```

12.10.1. Pulsos no bloqueantes

`halPulseSwitch()` usa `pulseStart()` que genera un **pulso no bloqueante** (por defecto 50 ms) y requiere que `pulseUpdate()` se llame en el lazo principal.

Acciones típicas de evento:

- bocina / efecto de sonido corto
- disparador “tomar foto” para una cámara
- resetear un subsistema (re-zero de IMU, línea de reset de controlador)
- relé momentáneo (pestillo de puerta, golpe de actuador)

12.11. Datos de Telemetría Enviados a la Aplicación Android

La telemetría es generada por:

```
telemetry.cpp
telemetry_source.cpp
```

El diseño está intencionalmente dividido:

- **telemetry.cpp** construye y envía paquetes (solo escritura).
- **telemetry_source.cpp** provee valores (lecturas de hardware o valores inyectados por debug).

12.11.1. Tipos de telemetría y tasas (implementado)

- **Indicadores:** cada 500 ms ($\approx 2\text{ Hz}$).
- **Gráficas (plots):** cada 50 ms ($\approx 20\text{ Hz}$).
- **Paneles:** retransmitidos brevemente cuando los valores cambian (ventana de reenvío para robustez).

12.11.2. Config-on-connect

En una nueva conexión, el ESP32 envía un **paquete de configuración** que incluye nombres de gráficas/paneles/indicadores (p. ej., Volts, Amps, RPMs, Left, Right, Throttle, Battery). Esto permite que la app Android etiquete elementos de UI automáticamente.

12.12. Ejemplo 7: Telemetría de Batería como Indicador

El paquete de indicadores del firmware incluye:

- analogValue (0–100)
- batteryLevel (0–100)
- digitalMask (bitfield)

En modo normal, la batería se lee desde un pin ADC y se mapea a un porcentaje (ver `hwReadIndicatorBattery()` en `input_hw.cpp`):

```
return map(analogRead(PIN_ANALOG_IND2), 0, 4095, 0, 100);
```

Nota práctica de cableado: el voltaje de batería debe escalarse con un divisor de voltaje para que el ADC del ESP32 nunca supere 3.3V.

12.13. Ejemplo 8: Paneles de Telemetría desde Entradas Analógicas

La telemetría de panel usa dos valores de 16 bits (`leftPanelValue`, `rightPanelValue`). La implementación de hardware por defecto lee valores analógicos y los escala de 0..4095 a 0..65535:

```
uint16_t hwReadNumeric1() {
    uint32_t v = analogRead(PIN_NUMERIC1);
    return (uint16_t)((v * 65535UL) / 4095);
}
```

Esto es adecuado para:

- sensores de distancia con salida analógica
- realimentación con potenciómetro
- señales de sensor escaladas (después de acondicionamiento)

12.14. Ejemplo 9: Señales de Gráfica (0..255 Gráficas Rápidas)

Las gráficas transmiten tres valores de 8 bits (0..255) a $\approx 20\text{ Hz}$. La implementación de hardware por defecto lee valores ADC y los mapea a 0..255:

```
uint8_t hwReadPlot1() {
    return map(analogRead(PIN_NUMERIC1), 0, 4095, 0, 255);
}
uint8_t hwReadPlot2() { \
    return map(analogRead(PIN_NUMERIC2), 0, 4095, 0, 255);
}
uint8_t hwReadPlot3() {
    return map(analogRead(PIN_ANALOG_IND2), 0, 4095, 0, 255);
}
```

Ideas prácticas para gráficas:

- tendencia de voltaje de batería (caída bajo aceleración)
- tendencia de sensor de corriente (detectar atascos)
- tendencia de velocidad derivada de RPM/encoder

12.15. Ejemplo 10: Indicadores de Máscara Digital

La telemetría de indicadores incluye un byte de máscara digital que puede representar hasta 8 señales on/off. En `input_hw.cpp`, se construye una máscara a partir de lecturas GPIO de indicadores (o desde entradas del MCP23017 en modo MCP).

Esto es útil para mostrar de forma compacta:

- finales de carrera
- banderas de falla
- sensores de parachoques
- señales de “armado” / “listo”

12.16. Resumen

El sistema soporta comunicación bidireccional:

- **Android** → **ESP32**: estado RC continuo + eventos discretos.
- **ESP32** → **Android**: paneles, indicadores y gráficas con configuración automática de etiquetas.

Al combinar entradas de control (sticks, perillas, switches, eventos) con retroalimentación de telemetría en vivo, puedes construir robots y sistemas embebidos que sean **fáciles de operar** y **fáciles de depurar en tiempo real**.

Capítulo 13

Capa de Control y Abstracción de Hardware

13.1. Propósito de Este Capítulo

Este capítulo documenta cómo el firmware **MICSBOL/ESP32_BT_Controller** separa:

- **la lógica de control** (qué debería hacer el robot cuando el usuario mueve un stick), y
- **la conducción del hardware** (cómo el ESP32 escribe señales PWM/GPIO/I²C).

El repositorio implementa esta separación principalmente mediante:

- `control.cpp / control.h` — la capa de control
- `hal_output.cpp` — la capa de abstracción de hardware (HAL)
- `pins.h` — el mapeo de hardware utilizado por el HAL
- `pulse.cpp / pulse.h` — salidas de pulso no bloqueantes usadas por acciones de evento

13.2. Arquitectura en Capas (Vista de Alto Nivel)

El firmware usa un flujo en capas desde la app Android hasta el hardware físico:

1. **La app Android** transmite paquetes de estado RC y paquetes de evento por Bluetooth (SPP).
2. **Capa receptora** (`receiver.cpp`) decodifica paquetes y actualiza:
 - `rcStatePacket` para valores de control continuo
 - `rcEventPacket` para eventos de botones de una sola vez
3. **Capa de control** (`control.cpp`) interpreta `rcStatePacket` y llama funciones del HAL.
4. **HAL** (`hal_output.cpp`) realiza las operaciones reales de GPIO/PWM/I²C.

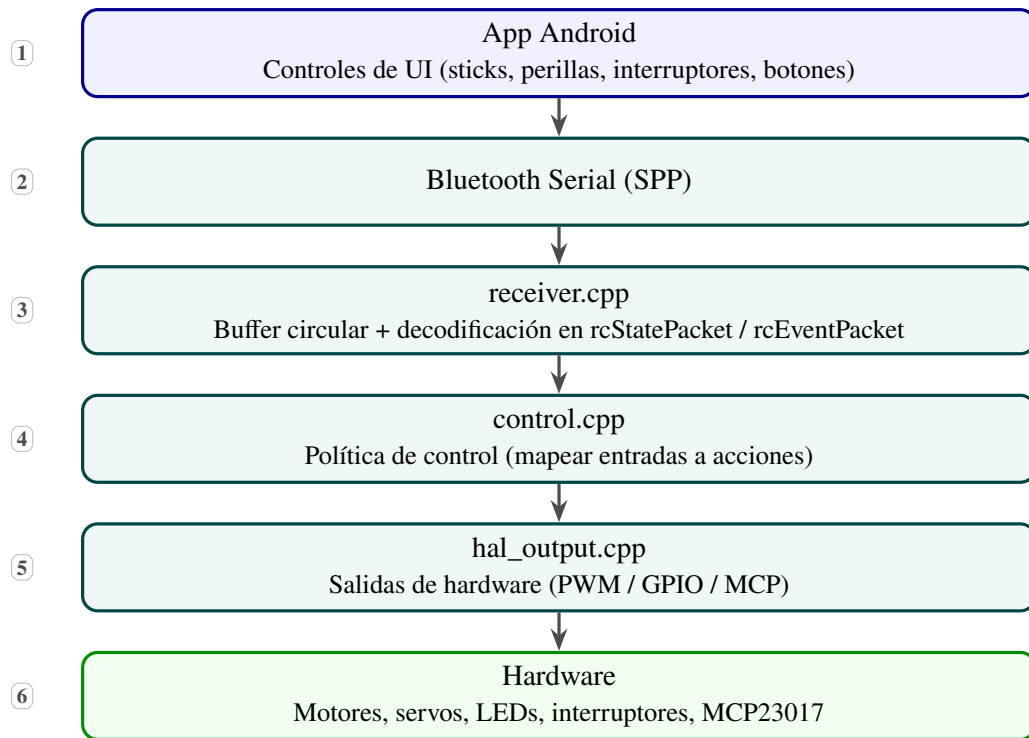


Figura 13.1: Flujo de datos de control desde Android hasta el hardware del ESP32.

13.3. Paquetes de Control y Estructuras de Estado

Los valores de control se almacenan en `rcStatePacket` (definido en `packets.h`). Los campos incluyen:

- `leftStickX`, `leftStickY`
- `rightStickX`, `rightStickY`
- `leftKnob`, `rightKnob`
- `switches` (máscara de bits)

En la lógica actual del firmware, los canales de sticks y perillas se tratan como valores de 12 bits (rango nominal 0 a 4095).

Las pulsaciones de botones se entregan mediante `rcEventPacket` (también en `packets.h`) y se manejan con `controlHandleEvent(eventId)`.

13.4. Capa de Control (`control.cpp`)

13.4.1. Responsabilidades

La capa de control es responsable de:

- mapear sticks y perillas a comandos de actuadores,
- seleccionar qué función del HAL llamar para cada salida lógica,
- decodificar la máscara de bits de switches a estados por salida,

- manejar eventos discretos de botones disparando salidas de pulso.

13.4.2. Bucle principal: `controlUpdate()`

La función principal de control es:

```
controlUpdate()
```

En la implementación del repositorio, `controlUpdate()` lee valores desde `rcStatePacket` y llama a:

- `halSetSteering(...)`
- `halSetMotorLeft(...)`
- `halSetMotorRight(...)`
- `halSetCameraPan(...)`
- `halSetLed(...)`
- `halSetBuzzer(...)`
- `halSetSwitch(index, state)`

Un helper clave es la función de mapeo de servo en `control.cpp`:

```
static uint32_t mapServo(uint16_t v) {
    return map(v, 0, 4095, 205, 410);
}
```

13.4.3. Decodificación de máscara de bits de switches

Los switches se transmiten como un solo byte `rcStatePacket.data.switches`. La capa de control convierte bits en salidas:

```
for (int i = 0; i < 6; i++) {
    halSetSwitch(i, (sw >> i) & 1);
}
```

13.4.4. Manejo de eventos: `controlHandleEvent(eventId)`

Los eventos discretos (pulsaciones de botones) se manejan mediante:

```
controlHandleEvent(byte eventId)
```

En el repositorio, los eventos se mapean a salidas de pulso mediante `halPulseSwitch(index)`. El mapeo depende del modo de proyecto (GPIO directos vs modo de expensor MCP).

13.5. Capa de Abstracción de Hardware (`hal_output.cpp`)

13.5.1. Responsabilidades

El HAL implementa los *mecanismos* para accionar el hardware:

- salida PWM mediante ESP32 LEDC usando `ledcWrite(...)`
- salida GPIO mediante `digitalWrite(...)`
- salida de expensor MCP23017 mediante `mcpWriteOutput(...)` (I²C)
- pines de dirección de motor y un mapeo con banda muerta
- rampa de velocidad de motor usando un callback periódico de timer
- salidas de pulso no bloqueantes usando el módulo de pulsos

13.5.2. Salidas PWM (ejemplos)

Ejemplos de `hal_output.cpp`:

```
void halSetSteering(uint32_t value) { ledcWrite(PIN_PWM_STICK_LX, value); }
void halSetLed(uint32_t value)      { ledcWrite(PIN_PWM_KNOB_L, value); }
void halSetBuzzer(uint32_t value)   { ledcWrite(PIN_PWM_KNOB_R, value); }
```

13.5.3. Control de motor: dirección + banda muerta + rampa

El control de motor en el HAL usa:

- pines de dirección `PIN_MD_LEFT` y `PIN_MD_RIGHT`
- pines de salida PWM `PIN_PWM_STICK_LY` (izquierdo) y `PIN_PWM_STICK_RY` (derecho)

El HAL interpreta valores de motor con una banda muerta alrededor del centro, y hace una rampa del PWM real del motor hacia un valor objetivo usando una función de actualización accionada por timer.

13.5.4. Salidas de switches: GPIO directos vs MCP23017

El HAL proporciona una interfaz uniforme:

```
halSetSwitch(uint8_t index, bool state)
```

pero la implementación depende del modo de compilación:

- En modo GPIO directo, se escribe un pequeño arreglo de pines ESP32 usando `digitalWrite(...)`.
- En modo MCP, la salida se escribe usando `mcpWriteOutput(...)` sobre I²C.

13.5.5. Salidas de pulso (acciones momentáneas)

Las salidas de pulso se disparan mediante `halPulseSwitch(index)` y se implementan usando:

- `pulseStart(pin, durationMs)` para iniciar un pulso
- `pulseUpdate()` para terminarlo después de la duración solicitada

Este diseño no es bloqueante, por lo que el bucle de control puede seguir ejecutándose mientras los pulsos están activos.

13.6. Mapeo de Pines (`pins.h`) y Portabilidad

Los mapeos de hardware están centralizados en `pins.h`. Al adaptar el firmware a nuevo hardware:

1. actualiza `pins.h` para que coincida con tu cableado,
2. ajusta `hal_output.cpp` si tu driver de motor o la lógica de salida difiere,
3. mantén `control.cpp` estable como “política” siempre que sea posible.

13.7. Resumen

El repositorio implementa una separación clara entre:

- **Política:** `control.cpp` interpreta `rcStatePacket` y decide qué acciones tomar.
- **Mecanismo:** `hal_output.cpp` acciona hardware PWM/GPIO/I²C para realizar esas acciones.

Esto hace que el firmware sea más fácil de portar entre diferentes robots y esquemas de cableado, manteniendo la base de código organizada y mantenible.

Capítulo 14

Diagrama de Cableado Explicado (Referencia del Repositorio)

14.1. Diagrama (Incluido)

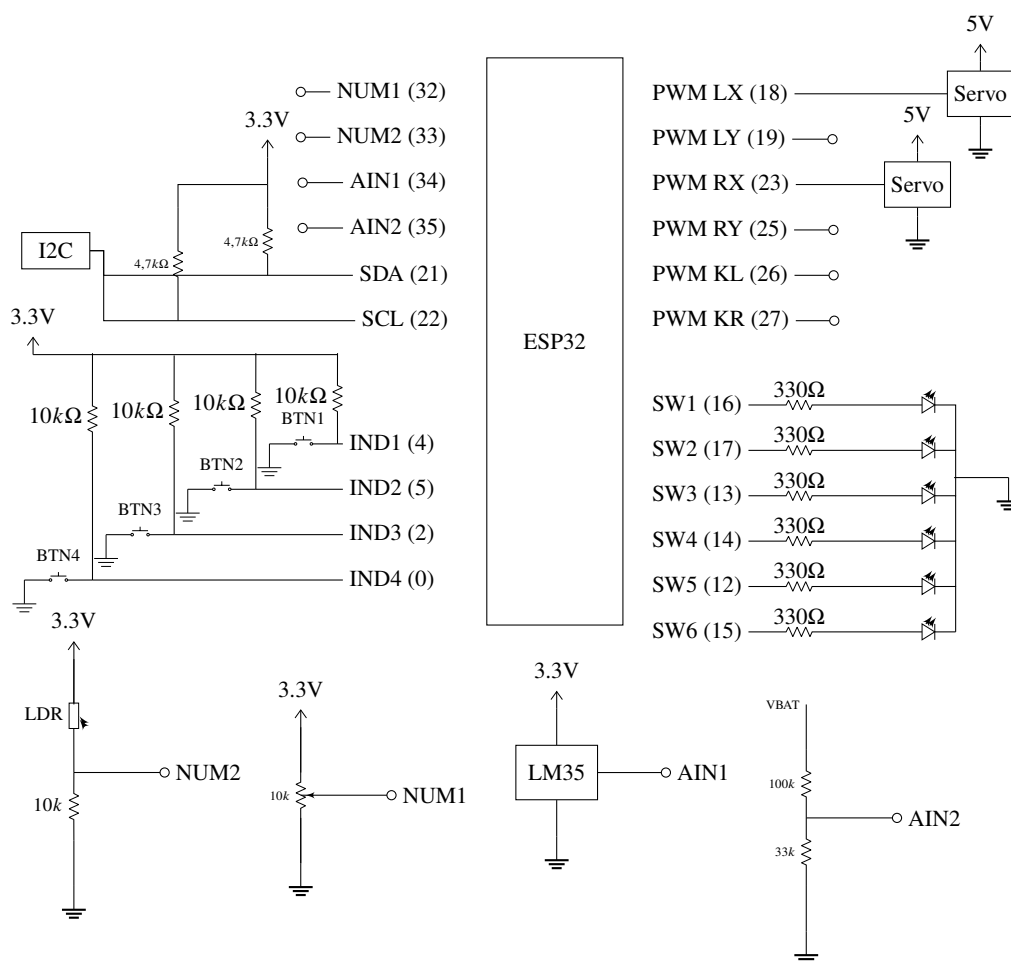


Figura 14.1: Asignación sugerida de pines del ESP32 (parte 1).

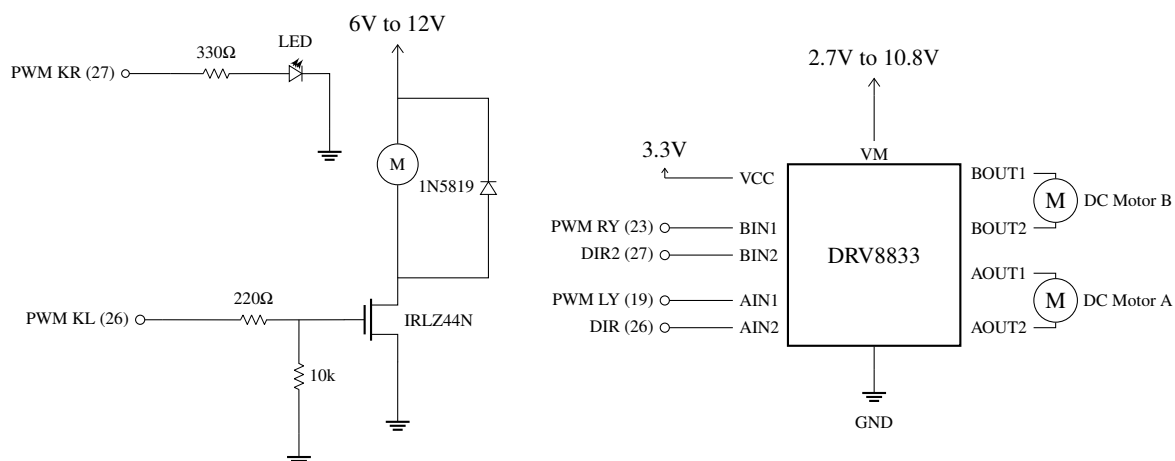


Figura 14.2: Asignación sugerida de pines del ESP32 y conexiones de periféricos para la configuración FULL_RC_MODE. Los circuitos mostrados son ejemplos y pueden modificarse según los requisitos específicos de la aplicación del usuario.

14.2. Resumen Rápido

La Figura 14.2 es una referencia práctica de cableado para el proyecto **MICSBOL/ESP32_BT_Controller**. Muestra:

■ Entradas (lado izquierdo):

- Bus I2C (SDA/SCL) para sensores o expansión
- Entradas analógicas (NUM1/NUM2/AIN1/AIN2) usadas como fuentes de telemetría
- Entradas de indicadores digitales (IND1–IND4) usadas para construir una máscara de estado

■ Salidas (lado derecho):

- Salidas PWM para servos, motores, brillo de LED y zumbador
- Salidas de switches (SW1–SW4 en modo directo, SW1–SW6 en modo MCP)
- Salidas de pulso/evento usadas para acciones momentáneas (eventos de botón)

En términos de código:

- `control.cpp` mapea campos de `rcStatePacket` a acciones (dirección, motores, paneo, LED, zumbador, switches).
- `hal_output.cpp` realiza las escrituras reales de hardware (PWM vía `ledcWrite`, GPIO vía `digitalWrite`, salidas MCP vía `mcpWriteOutput`, pulsos vía `pulseStart`).
- `input_hw.cpp` lee pines ADC e indicadores (usados como fuentes de telemetría).
- `pins.h` define el mapeo de GPIO (y cambia dependiendo del modo de proyecto).

14.3. Explicación Directa de Cada Grupo (Qué hace)

14.3.1. I2C (SDA 21, SCL 22)

Se usa para conectar dispositivos I2C externos. En el repositorio, el mismo bus también se usa en la configuración `FULL_RC_MODE_MCP` para comunicarse con un expensor MCP23017 (dirección `0x20` en `pins.h`).

14.3.2. Entradas analógicas de telemetría (NUM1/NUM2/AIN1/AIN2)

Estas son entradas ADC leídas en `input_hw.cpp` usando `analogRead(...)`. Están pensadas para fuentes de telemetría como voltaje de batería (a través de un divisor), sensores de temperatura, sensores de luz, potenciómetros, etc.

14.3.3. Entradas de indicadores digitales (IND1–IND4)

Estas son entradas digitales (GPIO) que representan estados on/off (fin de carrera, línea de falla, bumper, etc.). En `input_hw.cpp` se combinan en una máscara de bits para que múltiples estados puedan enviarse a la UI Android.

14.3.4. Salidas PWM (PWM LX/LY/RX/RY/KL/KR)

Las salidas PWM son accionadas por `hal_output.cpp` usando `ledcWrite(...)` y se alimentan con valores desde `control.cpp`. Usos típicos en el concepto de cableado del repositorio:

- servos (dirección, paneo de cámara)
- PWM de velocidad de motor
- brillo de LED
- intensidad del zumbador

14.3.5. Salidas de switches (SW1–SW6)

Estas son salidas digitales controladas por bits de switch recibidos desde la app Android (`control.cpp` llama a `halSetSwitch(index, state)`).

- El modo directo proporciona 4 salidas físicas (PIN_SW1..PIN_SW4).
- El modo MCP proporciona 6 salidas mediante MCP23017 (GPB0_SW1..GPB5_SW6).

14.3.6. Salidas de pulso/evento

Los eventos de botones Android se manejan por `controlHandleEvent(eventId)` en `control.cpp`, lo que dispara pulsos momentáneos mediante `halPulseSwitch(index)`. Los pulsos se implementan de forma no bloqueante en `pulse.cpp`. Estas salidas son ideales para disparos (bocina, pestillo, reset) más que para estados continuos.

14.4. Resumen (Un Párrafo)

El diagrama de cableado agrupa las conexiones del ESP32 exactamente de la manera en que el código del repositorio está organizado: `control.cpp` decide qué significa cada entrada, `hal_output.cpp` acciona salidas PWM y GPIO, `input_hw.cpp` lee entradas analógicas y de indicadores para telemetría, y `pins.h` define el mapeo de GPIO. Al mantener estas responsabilidades separadas, puedes adaptar el hardware (cambiar números de pin, agregar dispositivos I2C, cambiar a MCP23017) sin reescribir la lógica general de control.